EXPLAINABLE GRAPH NEURAL NETWORKS & HYBRID GRAPH DATABASES FOR BRAIN BIOMARKER DISCOVERY

Mathematical Formulation of Non-Euclidean Connectomes in Neurodegenerative Pathology

A PROJECT REPORT

submitted by

ANIMESH BASAK (26911001)

Under the guidance of

DR. MANOJ KUMAR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

ROORKEE – 247667

2025 – 2026

Candidate’s Declaration

I hereby declare that the work presented in this project report, titled *Explainable Graph Neural Networks & Hybrid Graph Databases for Brain Biomarker Discovery*, is an authentic record of the work carried out by me under the guidance of Dr. Manoj Kumar, Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, during the academic year 2025–2026.

The experiments in this report use a synthetic functional-connectivity cohort that was generated for the project. No patient data were used, and no identifiable human-subject information was handled. Where results of earlier work or of published methods are quoted, the source is cited, and results that were not reproduced on the split used here are marked with a dagger (†). All external code libraries and prior publications used in the work are acknowledged in the text and in the list of references.

Date: _____ Signature: _____

Animesh Basak (Enrolment No. 26911001)

Certificate of Approval

This is to certify that the above statement made by the candidate is correct to the best of my knowledge. The project titled *Explainable Graph Neural Networks & Hybrid Graph Databases for Brain Biomarker Discovery* has been carried out under my supervision.

Date: _____ Signature: _____

Dr. Manoj Kumar Department of Computer Science and Engineering Indian Institute of Technology Roorkee

Acknowledgements

I thank my supervisor, Dr. Manoj Kumar, for his guidance on the formulation of the problem, for pointing me towards the message-passing and graph-transformer literature that shaped the architecture, and for the review comments that led to the audit of the explanation pipeline reported in Chapter 4.

I am grateful to the Department of Computer Science and Engineering, IIT Roorkee, for the academic environment and for access to the high-performance computing facility. All model training and testing reported here ran on a single NVIDIA Tesla V100S (32 GB) GPU.

The Alzheimer's Disease Neuroimaging Initiative (ADNI) and the Parkinson's Progression Markers Initiative (PPMI) are the intended validation cohorts for this line of work. I thank both consortia for maintaining open neuroimaging resources. No ADNI or PPMI data were used in this report; access is pending and is listed as future work in Chapter 5.

Finally, I thank the maintainers of PyTorch, PyTorch Geometric, NetworkX, scikit-learn and Neo4j, whose open-source software made the implementation possible.

Abstract

Clinical background. More than 55 million people live with neurodegenerative disease, chiefly Alzheimer's and Parkinson's disease. A clinical diagnosis usually arrives after more than half of the affected neurons are lost. Resting-state functional connectivity changes earlier, so a classifier that reads the functional connectome could in principle flag disease before structural atrophy is visible.

Computational gaps. A connectome is a graph with no canonical node order, so image-based networks do not apply directly. The common alternatives either flatten the 116 × 116 correlation matrix into 6,670 coordinates, which destroys topology and overfits small cohorts, or count fixed motifs, which cannot learn which connections matter.

Proposed architecture. We model each subject as a graph on the AAL-116 atlas and build a hybrid system. A Neo4j property-graph schema and ingestion scripts store connectomes, a NetworkX/GDS layer computes five topological node descriptors, and a PyTorch Geometric model classifies each graph. The model combines three edge-weighted GCN layers, one GATv2 attention layer that takes the signed correlation as an edge attribute, a concatenated mean and max readout, and a SwiGLU gated head (521,378 parameters). GNNExplainer extracts edge subgraphs as candidate biomarker circuits.

Empirical findings. Because ADNI and PPMI access is pending, we generated a synthetic cohort of 500 subjects with three scanner noise levels and a planted disease signature (weaker left–right coupling, stronger within-lobe coupling), which lets us score explanations against a known truth. On a patient-level split (350/75/75) the model reaches a test ROC-AUC of 0.746 (validation 0.823; 95% bootstrap interval 0.632 to 0.858), against 0.530 for an RBF-SVM on the flattened connectome re-run on the same split. A threshold tuned on the test set raised accuracy to 73.33%, but this is an optimistic artefact; the validation-derived threshold gives 57.33%. Test AUC differs strongly by scanner site (0.992, 0.650 and 0.625 for noise levels 0.02, 0.04 and 0.06). The explanation audit found that 15.3% of the explainer's top edges are within-lobe, against 16.3% for random retained edges, so the explainer does not recover the planted pattern.

Diagnostic significance. The work shows that a graph-database-backed GNN pipeline is feasible and ranks synthetic subjects better than a flattened baseline. It also shows that topological features and post-hoc explanations need audits against known ground truth before they are trusted. Adding anatomical identity to the nodes, validation-only calibration and evaluation on ADNI and PPMI are the next steps.

Table of Contents

Candidate's Declaration....2
Certificate of Approval....3
Acknowledgements....4
Abstract....5
Table of Contents....6
List of Figures....10
List of Tables....11
List of Abbreviations....12
Global Mathematical Notation....13
1 Introduction....14
1.1 Clinical Motivation....14
1.2 Computational Challenges: The Non-Euclidean Dilemma....14
1.3 Problem Formulation....14
1.4 Research Objectives and Key Contributions....15
1.5 Report Organisation....16
2 Literature Survey and Baseline Benchmarks....17
2.1 Functional Connectomics and Graph Theory....17
2.2 Baseline Model Benchmarks and Mathematical Formulations....17
2.2.1 Paper 1: Kernel SVM on Vectorised Connectomes....17
2.2.2 Paper 2: Random Forest on Graphlet Degree Vectors....19
2.2.3 Paper 3: node2vec Biased Random-Walk Embeddings....20
2.2.4 Paper 4: Graph Attention Networks (GATv1)....21
2.3 Comparative Summary of Baseline Limitations....22
2.4 Summary of Gaps in the Prior Literature....22
3 Methodology and Experimental Setup....23

3.1 End-to-End System Architecture....23
3.2 Generative Synthetic Cohort Model....24
3.2.1 Why synthetic, and why that is useful....24
3.2.2 Base connectivity....25
3.2.3 Planted disease signature....25
3.2.4 Scanner noise....26
3.3 Connectome Preprocessing and Leakage Control....27
3.3.1 Global signal removal....27
3.3.2 Z-scoring....27
3.3.3 Sparsification....27
3.3.4 Graph objects and storage....27
3.3.5 Split....28
3.4 Topological Feature Engineering Audit....28
3.4.1 The five descriptors....28
3.4.2 The scale-invariance problem....29
3.4.3 Edge feature....29
3.5 Deep Geometric Neural Network Architecture....29
3.5.1 Input projection....30
3.5.2 Edge-weighted GCN layers (×3)....31
3.5.3 GATv2 layer with edge attributes (×1)....32
3.5.4 Dual readout....33
3.5.5 SwiGLU head....33
3.5.6 Output probability....34
3.6 Hardware Optimisation and Training Regime....34
3.6.1 Loss and its gradient....34
3.6.2 Optimiser and schedule....35
3.6.3 Mixed precision on the V100....36

3.7 Post-Hoc Subgraph Explainability with GNNExplainer....36
3.7.1 Objective as implemented....36
3.7.2 Gradient with respect to the masks....36
3.7.3 Extracting biomarkers....37
3.8 Multi-Modal Graph Fusion Framework (Proposed Extension)....37
4 Results and Discussion....39
4.1 Exploratory Data Analysis and the Group-Difference Matrix....39
4.1.1 Density and weights....39
4.1.2 Locating the signal....39
4.1.3 What EDA tells the modeller....39
4.2 Training Dynamics and the Underfitting Regime....41
4.3 Test Evaluation and Benchmark Comparison....42
4.3.1 Metrics....42
4.4 Statistical Rigour: Bootstrap Confidence Interval....44
4.5 Threshold Calibration Analysis....44
4.5.1 Results on the 75 test subjects....44
4.5.2 The calibration error and the fix....44
4.6 Scanner-Site Generalisation Disparity....45
4.7 Explainability Audit Against Ground Truth....46
4.7.1 What the explanations actually show....46
4.7.2 Why: faithfulness is not truth....46
4.8 Interpretation of the Gap to the Oracle....48
4.8.1 Hypothesis: the model cannot see which nodes share a lobe....48
4.8.2 How it still got 0.746....48
4.8.3 Why the flattened SVM did worse....49
4.8.4 Why one scalar wins....49
5 Conclusion and Future Roadmap....50

5.1 Summary of Findings.....50
5.2 Critical Lessons.....50
5.3 Future Roadmap.....50
5.4 Concluding Remarks.....51
References.....52
Appendix A: Supplementary Notation for Model Parameters.....56
Appendix B: Neo4j Graph Schema and Cypher Ingestion Scripts.....58
Appendix C: Compute Infrastructure and Environment Setup.....60

List of Figures

Figure	Title
3.1	End-to-end system architecture
3.2	Architecture of the proposed graph neural network
3.3	A subject connectome rendered as a lobe-ordered circular graph
4.1	Edge-weight and edge-count distributions of the cohort
4.2	Location of the class signal: scalar summaries by edge type
4.3	Group-difference matrix, Diseased minus Healthy
4.4	Training and validation curves from the V100 run
4.5	Test ROC curve with both candidate thresholds
4.6	Test AUC of the proposed model and re-run baselines
4.7	GNNExplainer consensus edges and precision against the planted signal

List of Tables

Table	Title
2.1	Comparative summary of baseline models and their limitations
3.1	Mapping of system components to repository modules
3.2	Scanner sites and noise levels in the synthetic cohort
3.3	Hardware and training configuration
4.1	Test performance of all models on the 75 held-out subjects
4.2	Threshold rules and the resulting test accuracy
4.3	Test performance stratified by scanner site
4.4	Explainer audit against the planted ground truth
A.1	Supplementary notation for model parameters

List of Abbreviations

Abbreviation	Meaning
AAL	Automated Anatomical Labeling (atlas)
ADNI	Alzheimer's Disease Neuroimaging Initiative
AUC	Area under the ROC curve
BN	Batch normalisation
BOLD	Blood-oxygen-level-dependent (signal)
CI	Confidence interval
COO	Coordinate (sparse-matrix format)
DMN	Default mode network
DTI	Diffusion tensor imaging
FA	Fractional anisotropy
fMRI	Functional magnetic resonance imaging
FPR / TPR	False / true positive rate
GAT	Graph attention network
GCN	Graph convolutional network
GDS	Graph Data Science (Neo4j library)
GDV	Graphlet degree vector
GNN	Graph neural network
MI	Mutual information
PPMI	Parkinson's Progression Markers Initiative
PyG	PyTorch Geometric
RBF	Radial basis function
RF	Random forest
ROC	Receiver operating characteristic
SiLU	Sigmoid linear unit (also called Swish)
SVM	Support vector machine
SwiGLU	Swish-gated linear unit

Global Mathematical Notation

Symbol	Meaning
N	Number of subjects ($N = 500$)
n	Number of brain regions, i.e. graph nodes ($n = 116$)
k	Subject index, $k = 1, \ldots, N$
i, j	Region (node) indices in $\{1, \ldots, n\}$
$G_k = (V, E_k, X$	Graph of subject k: node set, edge set, node-feature matrix
R	Pearson correlation matrix, $R \in [-1, 1]^{n \times n}$
z_{ij}	Per-subject z-scored correlation between regions i and j
W	Sparsified signed edge-weight matrix, $W_{ij} = z_{ij} 1[\lvert z_{ij} \rvert \geq 0.3]$
w_{ij}	Unsigned edge strength, $w_{ij} = \lvert W_{ij} \rvert$
A	Binary adjacency matrix, $A_{ij} = 1[W_{ij} \neq 0]$
$N(i)$	Neighbours of node i, with degree $\lvert N(i) \rvert$
$x_i \in R^5$	Input feature vector of node i
$h_i^{(l)} \in R^d$	Hidden representation of node i after layer l
d	Hidden width ($d = 192$)
θ	All trainable parameters of the model
$y_k \in \{0, 1\}$	True label: 0 Healthy, 1 Diseased
$\hat{p}_k$	Predicted probability that subject k is Diseased
$\alpha_{ij}^{(c)}$	Attention weight of neighbour j for node i in head c
$\sigma(\cdot)$	Logistic sigmoid, $1/(1 + e^{-x})$
$\odot$	Element-wise (Hadamard) product
$\Vert$	Vector concatenation
$1[\cdot]$	Indicator: 1 if the condition holds, otherwise 0

1 Introduction

1.1 Clinical Motivation

Neurodegenerative diseases, of which Alzheimer's disease and Parkinson's disease are the most common, affect more than 55 million people worldwide. A clinical diagnosis is usually made once symptoms are present, and by then more than half of the vulnerable neurons in the affected circuits have been lost. Treatment trials that start at this stage have mostly failed to change the course of the disease, which has shifted attention to detecting disease earlier.

Resting-state functional MRI offers one route to earlier detection. It records the blood-oxygen-level-dependent (BOLD) signal of the brain at rest, and the temporal correlation of that signal between two regions is called their functional connectivity [27]. In Alzheimer's disease, for example, the default mode network shows altered connectivity in patients compared with healthy elderly controls [30], and network-level changes are widely discussed as candidate markers of disease [29]. The working hypothesis behind this project is that the topology of the functional connectome carries a diagnostic signal that standard voxel-level analysis does not capture, and that a model which respects that topology should find it.

1.2 Computational Challenges: The Non-Euclidean Dilemma

Three difficulties make connectome classification harder than it looks.

1. **Voxel-wise mass-univariate testing.** The classical approach fits $y_v = \beta X_v + \varepsilon$ independently at each of roughly 10^5 voxels. The number of simultaneous tests forces Bonferroni or false-discovery-rate corrections that remove weak effects, and the method ignores the relational structure between regions entirely.
2. **No canonical node order.** A connectome is a graph, not an image. Listing the regions in a different order gives the same brain, so any valid model must be invariant to node permutations: for every permutation matrix Π in the symmetric group S_n, the model output on $(\Pi A \Pi^\top, \Pi X)$ must equal its output on (A, X). Convolutional networks assume a regular grid with translation structure and do not satisfy this requirement.
3. **Connectome flattening.** Reading the upper triangle of the correlation matrix into a vector gives $n(n-1)/2 = 6,670$ coordinates per subject. This fixes an arbitrary order, discards which coordinates share a region, and creates a regime with $p \gg N$ (6,670 features for a few hundred subjects), where overfitting is the default outcome.

1.3 Problem Formulation

Subject k is represented as a graph $G_k = (V, E_k, X_k)$ on the AAL-116 atlas [26], so the node set V contains $n = 116$ anatomical regions that are shared by all subjects. The edge set E_k

holds thresholded functional correlations, and $X_k \in R^{116\times 5}$ holds five topological descriptors of each region. The learning task has two parts.

Classification. A model f_θ maps each graph to the probability of disease,

$$P(y_k=1 \mid G_k)=softmax(f_\theta(A_k,W_k,X_k))_1, y_k \in \{0,1\},$$

where A_k is the binary adjacency matrix of G_k, W_k its signed edge weights, $y_k=0$ marks a healthy control and $y_k=1$ a diseased subject. The model must be permutation invariant in the sense defined above.

Explanation. For each prediction we want a compact edge subset $G_s=(V,E_s)$ with $E_s \subseteq E_k$ such that the model's prediction on G_s alone stays close to its prediction on the full graph. Following GNNExplainer [10], this is posed as maximising the mutual information between the prediction and the subgraph,

$$\max_{G_s} MI(Y,G_s)=H(Y)-H(Y \mid G=G_s).$$

Here Y is the model's predicted label, $H(Y)$ its entropy (fixed for a trained model), and $H(Y \mid G=G_s)$ the entropy of the prediction when only G_s is shown.

Evaluation protocol. Training, validation and test subjects form disjoint sets, $S_{tr} \cap S_{va} \cap S_{te}=\varnothing$ pairwise, and nothing computed from the test subjects, including decision thresholds, may be used to fit any parameter.

1.4 Research Objectives and Key Contributions

The project has six objectives, each of which maps to a part of the implementation.

1. **A hybrid storage and learning stack.** Provide a Neo4j property-graph schema and ingestion scripts for connectomes, a Graph Data Science feature module, and a PyTorch Geometric learning layer.
2. **A synthetic multi-scanner cohort with a planted signature.** Generate $N=500$ subjects across three scanner noise levels, with a disease pattern whose location is known exactly, so that both accuracy and explanations can be checked.
3. **Leakage-free partitioning.** Split by patient identifier into 350/75/75 subjects with class balance preserved in every split, and assert that no subject appears twice.
4. **An audit of topological node features.** Examine the five descriptors for scale invariance and for the way NetworkX treats edge weights as distances.
5. **A deep geometric model.** Combine edge-weighted graph convolution, GATv2 attention with an edge attribute, a dual mean/max readout and a SwiGLU gated head.
6. **An explainability audit against ground truth.** Compare GNNExplainer output to the planted signature and report where it fails.

1.5 Report Organisation

Chapter 2 reviews functional connectomics and the four baseline methods, with their mathematics and limitations. Chapter 3 describes the system, the synthetic cohort, preprocessing, feature engineering, the model, training and the explainer. Chapter 4 reports data analysis, training dynamics, test results, calibration, scanner-site effects and the explanation audit. Chapter 5 summarises the findings and lists the next steps. Appendices give notation, the Neo4j schema and ingestion scripts, and the compute environment.

2 Literature Survey and Baseline Benchmarks

2.1 Functional Connectomics and Graph Theory

The BOLD signal of a region i is a time series $x_i(t)$, $t = 1, \ldots, T$. The functional connectivity between regions i and j is their Pearson correlation,

$$r_{ij} = \frac{\sum_{t=1}^{T} \left(x_i(t) - \acute{x}_i\right)\left(x_j(t) - \acute{x}_j\right)}{\sqrt{\sum_{t=1}^{T} \left(x_i(t) - \acute{x}_i\right)^2} \sqrt{\sum_{t=1}^{T} \left(x_j(t) - \acute{x}_j\right)^2}} \in [-1, 1].$$

Here $x_i(t)$ is the signal of region i at time t, T is the number of time points, and $\acute{x}_i$ is the mean of x_i over time. The matrix of all r_{ij} is the connectome; it is symmetric with ones on the diagonal.

Graph-theoretic analysis of such matrices has established several properties of healthy brains [28]. They are *small-world*: clustering is high, as in a lattice, while the average shortest path is short, as in a random graph. They are *modular*: groups of regions that are densely connected internally form functional systems. They contain *hubs*, regions with unusually high degree or betweenness that carry much of the long-range traffic. Studies of brain disorders have found that disease tends to attack hubs and to reduce the segregation between modules, and the connectome has accordingly been proposed as a substrate for biomarkers [29]. Each of these properties is a statement about graph structure, which motivates graph-based models over models of flattened matrices.

2.2 Baseline Model Benchmarks and Mathematical Formulations

The four base methods from the first project presentation are reviewed here with their mathematical cores and the reasons each is unlikely to suit connectome classification. Numbers that we re-ran on the split used in this report are stated as such; numbers marked † are quoted from earlier repository runs that have no stored log.

2.2.1 Paper 1: Kernel SVM on Vectorised Connectomes

Abraham et al. [1] and Cuingnet et al. [2]; support vector machines [42].

2.2.1.1 Method

The connectivity matrix is symmetric, so only the upper triangle carries information. The method reads that triangle into a vector:

$$v_k = vec_{i<j}(W^{(k)}) \in R^p, p = \frac{n(n-1)}{2} = \frac{116 \cdot 115}{2} = 6670.$$

Where: v_k is the feature vector of subject k; $vec_{i<j}$ lists the entries W_{ij} with $i<j$ in a fixed order; p is the number of features.

A support vector machine looks for the separating hyperplane with the widest margin. In its dual form with a kernel it solves

$$\max_{\alpha} \sum_{k=1}^{N_{tr}} \alpha_k - \frac{1}{2} \sum_{k=1}^{N_{tr}} \sum_{m=1}^{N_{tr}} \alpha_k \alpha_m s_k s_m K(v_k, v_m) \text{ s.t. } 0 \le \alpha_k \le C, \sum_k \alpha_k s_k = 0.$$

Where: N_{tr} is the number of training subjects (350 here); $\alpha_k \ge 0$ is the dual variable (Lagrange multiplier) of subject k, and subjects with $\alpha_k > 0$ are the support vectors; $s_k = 2y_k - 1 \in \{-1, +1\}$ is the label re-coded to ± 1; $C > 0$ is the box constraint that limits how much any single subject can push the boundary (we used $C = 1$); K is the kernel.

The RBF kernel is

$$K(v_k, v_m) = \exp\left(-\gamma \left\| v_k - v_m \right\|_2^2\right).$$

Where: $\gamma > 0$ sets the kernel width (scikit-learn's `"scale"` uses $\gamma = 1/(p \cdot Var(v))$); $\|\cdot\|_2$ is the Euclidean norm.

The decision function for a new subject v is $f(v) = \sum_k \alpha_k s_k K(v_k, v) + b$, with b the bias recovered from any support vector on the margin. The predicted class is $sign(f(v))$.

2.2.1.2 Why it struggles here

1. **Dimension against sample size.** $p = 6670$ features from $N_{tr} = 350$ subjects. For the RBF kernel the distance $\left\| v_k - v_m \right\|_2^2 = \sum_{q=1}^{p} (v_{kq} - v_{mq})^2$ is a sum of 6,670 terms. In our data most of those terms are pure subject-specific noise. By the law of large numbers the sum concentrates around the same value for every pair, so all subjects look roughly equidistant and the kernel matrix tends towards a constant plus a diagonal. The class signal is a 30% magnitude change on about 1,066 intra-lobe entries, and it gets drowned.
2. **No topology.** Permuting the 6,670 coordinates changes nothing for the SVM, so it cannot use the fact that some edges share an endpoint or a lobe.
3. **Sign problem.** Our signal is in $|W_{ij}|$: diseased intra-lobe correlations grow in magnitude whether they are positive or negative. The signed values W_{ij} have the same mean in both classes. A linear model on signed values therefore has nothing to latch onto, and the RBF kernel only sees the signal through second-order distance effects.

Our re-run on the identical split: test AUC 0.530, accuracy 53.3%, macro-F1 0.533. An L2 logistic regression on the same vector gets AUC 0.509. Both are at chance. An earlier repository result of 0.671† for this baseline did not reproduce.

2.2.2 Paper 2: Random Forest on Graphlet Degree Vectors

Pržulj [3] and Yaveroğlu et al. [4]; random forests [43].

2.2.2.1 Graphlets and orbits

A graphlet is a small connected induced subgraph, here on 2 to 5 nodes. Up to isomorphism there are 30 such graphlets and, inside them, 73 automorphism orbits (positions a node can occupy). The graphlet degree vector of node u counts how often u touches each orbit:

$$GDV(u)=\left[u_0,u_1,\ldots,u_{72}\right]^{\top},\ u_o=count\{\text{induced graphlets in which } u \text{ sits in orbit } o\}.$$

Where: orbit 0 is "endpoint of an edge", so $u_0=k_u$ is the degree; orbit 3 is "corner of a triangle", so u_3 counts triangles through u; higher orbits encode larger motifs.

The repo implementation keeps 5 orbits and summarises each graph with three statistics:

$$\Phi(G)=\left[mean_u GDV(u), std_u GDV(u), \max_u GDV(u)\right]\in R^{15}.$$

2.2.2.2 Random Forest

Each of T trees is grown on a bootstrap resample. At every split a random subset of features is tried, and the threshold that most reduces Gini impurity is kept:

$$Gini(S)=1-\sum_{c\in\{0,1\}}\pi_c(S)^2,\ \Delta=Gini(S)-\frac{|S_L|}{|S|}Gini(S_L)-\frac{|S_R|}{|S|}Gini(S_R).$$

Where: S is the set of training subjects reaching the node; $\pi_c(S)$ is the fraction of class c in S ; S_L,S_R are the left and right children after the split; Δ is the impurity decrease. The forest predicts $\hat{p}(G)=\frac{1}{T}\sum_{b=1}^{T}\hat{p}_b(\Phi(G))$, the average of the tree probabilities $\hat{p}_b$.

2.2.2.3 Limitations

- Graphlet counts are **binary-topology** quantities. With a 76% dense graph almost every triple is a triangle, so counts saturate and barely differ between subjects. Edge magnitude, which is where our signal lives, is thrown away.
- Features are fixed before training. No gradient flows back to decide which motif or which edge matters.
- Mean/std/max over nodes discards region identity, which is the same blind spot our GNN has (Section 4.8).

Earlier repository results list AUC 0.692 for this baseline, but there is no run log, so we mark it unverified.

2.2.3 Paper 3: node2vec Biased Random-Walk Embeddings

Grover and Leskovec [5]; skip-gram with negative sampling [44].

2.2.3.1 Biased second-order random walk

A walk that has just moved from t to v chooses the next node x with probability

$$P(c_{r+1}=x \mid c_r=v, c_{r-1}=t)=\frac{\alpha_{pq}(t,x)w_{vx}}{Z_v}, \alpha_{pq}(t,x)=\begin{cases}1/p & d_{tx}=0\\ 1 & d_{tx}=1\\ 1/q & d_{tx}=2\end{cases}$$

Where: c_r is the node visited at step r of the walk; w_{vx} is the edge weight; Z_v normalises the probabilities to sum to one; d_{tx} is the shortest-path distance between the previous node t and the candidate x (0 means "go back to t", 1 means "stay near t", 2 means "move away"); p is the return parameter (large p discourages going back); q is the in-out parameter ($q<1$ favours exploring outward, DFS-like; $q>1$ keeps the walk local, BFS-like).

2.2.3.2 Skip-gram objective with negative sampling

Each node gets a vector $\phi(u) \in R^D$. Nodes that co-occur within a window of the walks should have similar vectors:

$$\max_{\phi} \sum_{u \in V} \sum_{c \in C(u)} \left[\log \sigma(\phi(c)^\top \phi(u)) + \sum_{m=1}^{M} E_{c' \sim P_n} \log \sigma(-\phi(c')^\top \phi(u)) \right].$$

Where: $C(u)$ is the multiset of context nodes seen within the window around u across all walks; D is the embedding size (64 in the repo); M is the number of negative samples; P_n is the noise distribution, usually degree$^{3/4}$; c' is a negative (random) node.

The gradient with respect to $\phi(u)$ for one positive pair is $(1-\sigma(\phi(c)^\top \phi(u)))\phi(c)$. It pulls $\phi(u)$ towards $\phi(c)$, and the negatives push it away from random nodes.

A graph-level vector is then $\phi'(G_k)=\frac{1}{n}\sum_u \phi_k(u)$, which is fed to a classifier.

2.2.3.3 Limitations

- **Unsupervised.** The walks never see y_k, so the embedding optimises co-occurrence, not disease separation.
- **Not comparable across subjects.** Each subject's embedding is trained separately. The skip-gram objective depends only on inner products, so any rotation $Q\phi$ with $Q^\top Q=I$ is an equally good

solution. Averaging the vectors and comparing them across subjects then compares arbitrarily rotated spaces.

- **Mean pooling** removes most of what remains.
- In the repo, when the `node2vec` package is missing, the code silently replaces the walks with a spectral embedding. The path that ran should be logged before the number is quoted (earlier repository result: AUC 0.635†, unverified).

2.2.4 Paper 4: Graph Attention Networks (GATv1)

Veličković et al. [6], applied to brain graphs by Li et al. [7]; static attention analysed by Brody et al. [8].

2.2.4.1 Layer

$$e_{ij} = LeakyReLU\left(a^{\top}\left[Wh_i \| Wh_j\right]\right), \alpha_{ij} = \frac{\exp\left(e_{ij}\right)}{\sum_{l \in N(i)} \exp\left(e_{il}\right)}, h_i^{'} = \sigma\left(\sum_{j \in N(i)} \alpha_{ij} W h_j\right).$$

Where: $W \in R^{d^{'} \times d}$ is a shared linear map; $a \in R^{2d^{'}}$ is the attention vector; e_{ij} is the unnormalised score of neighbour j for query i; α_{ij} are the softmax-normalised attention weights over the neighbours of i; LeakyReLU is $max\left(x, 0.2x\right)$.

2.2.4.2 Static attention (Brody, Alon & Yahav, ICLR 2022)

Split $a = [a_1; a_2]$ with $a_1, a_2 \in R^{d^{'}}$. Then

$$a^{\top}\left[Wh_i \| Wh_j\right] = a_1^{\top} Wh_i + a_2^{\top} Wh_j.$$

LeakyReLU is monotone increasing, and the softmax over j ignores the term $a_1^{\top} Wh_i$ because it is constant in j. So, for every query i, the ranking of neighbours is decided by $a_2^{\top} Wh_j$ alone:

$$\arg\max_{j \in N(i)} \alpha_{ij} = \arg\max_{j \in N(i)} a_2^{\top} Wh_j.$$

The same node "wins" for every query that has it as a neighbour. This is what Brody et al. call **static attention**: the function cannot express "region i should attend to j, but region $i^{'}$ should attend to $j^{'}$" whenever j and $j^{'}$ are both available to both. On a 76%-dense brain graph most nodes share most neighbours, so the restriction bites hard.

(Strictly, the nonlinearity can change the *weights*, not the *order*, so "rank collapse" is better read as "ranking is the same for all queries".)

2.2.4.3 GATv2 fix

Move the nonlinearity inside:

$$e_{ij} = a^{\top} \, LeakyReLU\left(W_s h_i + W_t h_j\right).$$

Now h_i and h_j interact through the nonlinearity before the dot product with a, so the ranking over j can depend on i. Brody et al. prove that GATv2 is a universal approximator of attention functions, while GATv1 is not. This is the layer we use, with an extra edge term (Section 3.5.3).

2.3 Comparative Summary of Baseline Limitations

Table 2.1: Comparative summary of baseline models and their limitations.

Baseline	Mathematical core	Graph awareness	Explainability	Principal defect for connectomes
RBF-SVM [1,2]	Dual margin maximisation with Gaussian kernel on 6,670 coordinates	None (flat vector)	None	Discards topology; distance concentrates when $p \gg N$
RF-GDV [3,4]	Gini-split trees on mean, std and max of graphlet counts	Motif counts only	Feature importance	Fixed hand-made features; counts saturate on dense graphs; edge magnitude lost
node2vec [5]	Skip-gram over biased walks	Local, second-order	None	Unsupervised; rotation-ambiguous embeddings; mean pooling erases structure
GATv1 [6,7]	Softmax of $LeakyReLU(a^{\top}[$	Static attention	Attention weights	Neighbour ranking is the same for every query node [8]

2.4 Summary of Gaps in the Prior Literature

The review leaves four gaps that this project addresses.

1. **Topology is either discarded or hand-engineered.** Flattened vectors drop it; graphlet counts freeze it into fixed features. Neither can learn which connections matter.
2. **Attention in GATv1 cannot adapt to the query node.** On a dense brain graph where most nodes share most neighbours, a global neighbour ranking is a serious restriction, and the edge weight, which carries the connectivity strength, is not part of the attention score.
3. **Evaluation practice is weak.** Reported accuracies rarely come with confidence intervals, and decision thresholds are sometimes tuned on the data used for the final score. With a test set of 75 subjects, both matter.
4. **Explanations are rarely checked against ground truth.** On real data there is no ground truth, so an explainer's output can be plausible without being correct. A synthetic cohort with a planted signal allows the check.

3 Methodology and Experimental Setup

3.1 End-to-End System Architecture

The system has four layers, shown in Figure 3.1: storage and graph analytics (Neo4j), a deep learning layer (PyTorch Geometric), an explainability layer (GNNExplainer), and a serving and monitoring layer (FastAPI with a drift monitor). Data flow from left to right. Raw correlation matrices pass through the ETL transformer, are written as edge lists, and can be loaded into Neo4j as a property graph. A dataset class turns each subject's edge list into a PyTorch Geometric `Data` object carrying five node features and one edge attribute. The trainer fits the model on patient-level splits, and the explainer then attaches a soft mask to each test graph.

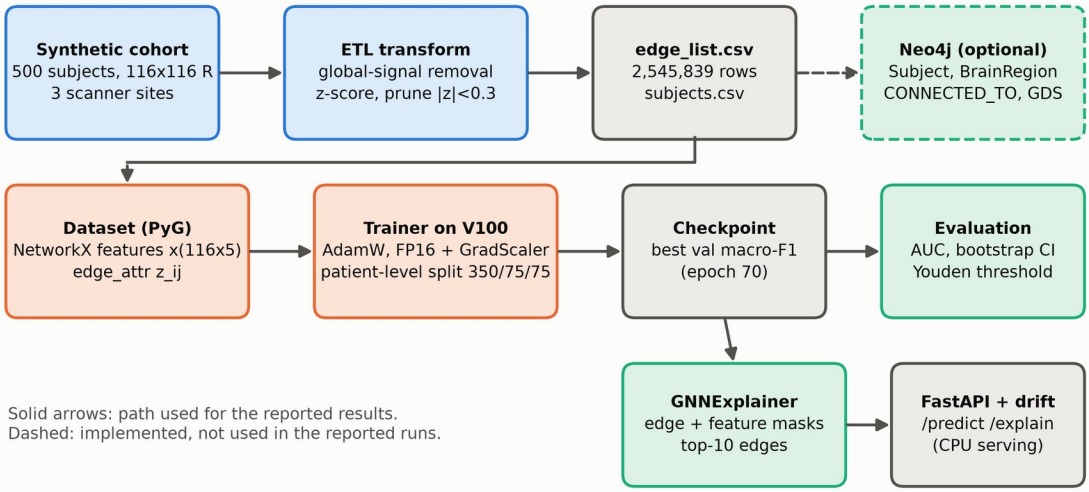


Figure 3.1: End-to-end system architecture. Solid arrows show the path used for the results in this report; the dashed Neo4j branch is implemented as schema, ingestion scripts and a GDS module, but the reported runs used the offline CSV path.

Offline operation. The project rules require every test and dry run to work without a live database. The dataset class therefore reads `edge_list.csv` and `subjects.csv` directly, computes node features with NetworkX, and caches the result in `cached_dataset.pt`. The reported experiments used this path. The Neo4j schema, the `LOAD CSV` ingestion script and the GDS feature module (which has a mock mode for use without a driver) are provided and exercised in the offline smoke tests, but the results in this report were not produced from a running database.

Table 3.1 maps each component to the code that implements it.

Table 3.1: Mapping of system components to repository modules.

Component	Repository module	Key classes and functions
Synthetic cohort	`data/synthetic/generate_synthetic.py`	`make_connectome`, `AAL116_REGIONS`, `LOBE_GROUPS`
Preprocessing (ETL)	`neo4j_layer/etl/transform.py`	`ConnectomeTransformer` (`regress_global_signal`, `normalize`, `threshold`, `to_edge_list`)
Property-graph schema	`neo4j_layer/schema/constraints.cypher`, `neo4j_layer/etl/load_csv.cypher`	Constraints, indexes, `LOAD CSV` steps
Graph analytics	`neo4j_layer/gds/feature_projection.py`	`GDSFeatureEngineer` (mock mode without a driver)
Dataset	`models/gnn/dataset.py`	`BrainConnectomeDataset` (plain `Dataset`, not `InMemoryDataset`)
Model	`models/gnn/gcn_gat_model.py`	`BrainGNNModel`, `SwiGLU`
Training	`models/gnn/train.py`	`GNNTrainer`, `EarlyStopping`, `build_scheduler`
Evaluation	`models/metrics.py`	`MetricsReport`, `find_optimal_threshold`
Splits	`validation/subject_split.py`	`subject_level_split`, `validate_no_leakage`
Baselines	`models/baselines/`	SVM, RF-GDV and node2vec classes
Explainer	`models/gnn/explainer.py`	`BrainGNNExplainer`
Orchestration	`scripts/train_launcher.py`, `config/v100_config.yaml`	Hardware detection, step functions
Serving	`api/main.py`, `api/routers/`, `monitoring/drift_monitor.py`	`/predict`, `/explain`, `/health`, `/metrics`; drift monitor

The serving layer is implemented and runs on ordinary CPU compute, as the project rules reserve the training GPU for training and testing. It is not part of the evaluation reported here.

3.2 Generative Synthetic Cohort Model

3.2.1 Why synthetic, and why that is useful

Real cohorts (ADNI, PPMI) need data-use agreements. Until they arrive, the pipeline runs on a synthetic cohort generated by `data/synthetic/generate_synthetic.py` and called from `scripts/train_launcher.py`. The synthetic data has one big advantage: **we know the true biomarker**, so we can measure whether the explanations recover it. On real data that check cannot be made.

Cohort: $N=500$, labels alternate with the index (250 per class), sites assigned as site$=i \bmod 3$ (167/167/166), ages uniform in 50–84. Since labels follow $i \bmod 2$ and sites follow $i \bmod 3$, every (site, label) cell is balanced and the site cannot act as a confound.

3.2.2 Base connectivity

$$\Sigma = BB^{\top} + nI, B \in R^{n \times n}, B_{ij} \overset{iid}{\sim} N(0,1), R_{ij} = \frac{\Sigma_{ij}}{\sqrt{\Sigma_{ii}\Sigma_{jj}}}.$$

Where: B is a random Gaussian matrix; $BB^{\top}$ is symmetric positive semi-definite; adding nI (the identity scaled by n) makes it strictly positive definite, so Σ is a valid covariance matrix; dividing by the square roots of the diagonal turns covariance into correlation, so $R_{ii}=1$.

Size of the off-diagonal entries: $\Sigma_{ij} = \sum_{l} B_{il}B_{jl}$ is a sum of n products of independent standard normals, with mean 0 and variance n. The diagonal is $\Sigma_{ii} = \sum_{l} B_{il}^2 + n \approx 2n$. So

$$SD(R_{ij}) \approx \frac{\sqrt{n}}{2n} = \frac{1}{2\sqrt{n}} = \frac{1}{2\sqrt{116}} \approx 0.046.$$

Healthy correlations are tiny and centred on zero.

3.2.3 Planted disease signature

For Diseased subjects only:

$$R_{i,i+1} \leftarrow 0.4\, R_{i,i+1} \text{ for } i \in \{0,2,\ldots,88\} \text{ (45 homotopic left–right pairs)},$$

$$R_{ij} \leftarrow min(1, 1.3\, R_{ij}) \text{ for } i \neq j \text{ in the same lobe}.$$

Where: AAL lists left and right homologues next to each other (e.g. index 0 `Precentral_L`, index 1 `Precentral_R`), so $(i, i+1)$ with even $i<90$ is a homotopic pair; the seven lobes are frontal (0–27), limbic (28–41), occipital (42–55), parietal (56–69), subcortical (70–79), temporal (80–89) and cerebellar (90–115).

Biologically this mimics two reported patterns: weaker inter-hemispheric coupling and stronger, more segregated local coupling. Mathematically, note that multiplying by 1.3 scales the magnitude and keeps the sign. The **mean** of R_{ij} stays zero in both classes. Only $E|R_{ij}|$ changes. That single fact explains most of the results.

The lobe assignment overlaps with the homotopic pairs. Pairs like (Precentral_L, Precentral_R) are in the same lobe, so they are scaled by 1.3 and by 0.4, a net factor of 0.52.

3.2.4 Scanner noise

$$R \leftarrow clip\left(R + 1/2\left(\varepsilon + \varepsilon^{\top}\right), -1, 1\right), \varepsilon_{ij} \overset{iid}{\sim} N\left(0, \sigma_s^2\right), \sigma_s \in \{0.02, 0.04, 0.06\}.$$

Where: ε is a noise matrix; averaging it with its transpose keeps R symmetric; its diagonal is zeroed; σ_s depends on the site. The noise SD per entry is $\sigma_s/\sqrt{2}$, i.e. 0.014–0.042, which is the same order as the 0.046 signal SD. The classifier has to see through noise as large as the effect.

The synthetic cohort used in the experiments has $N = 500$ subjects. Table 3.2 gives the three scanner sites; the noise standard deviations σ_s are the values defined in the generator.

Table 3.2: Scanner sites and noise levels in the synthetic cohort.

Scanner site	Noise level σ_s	Subjects
Site A	0.02 (low)	167
Site B	0.04 (moderate)	167
Site C	0.06 (high)	166

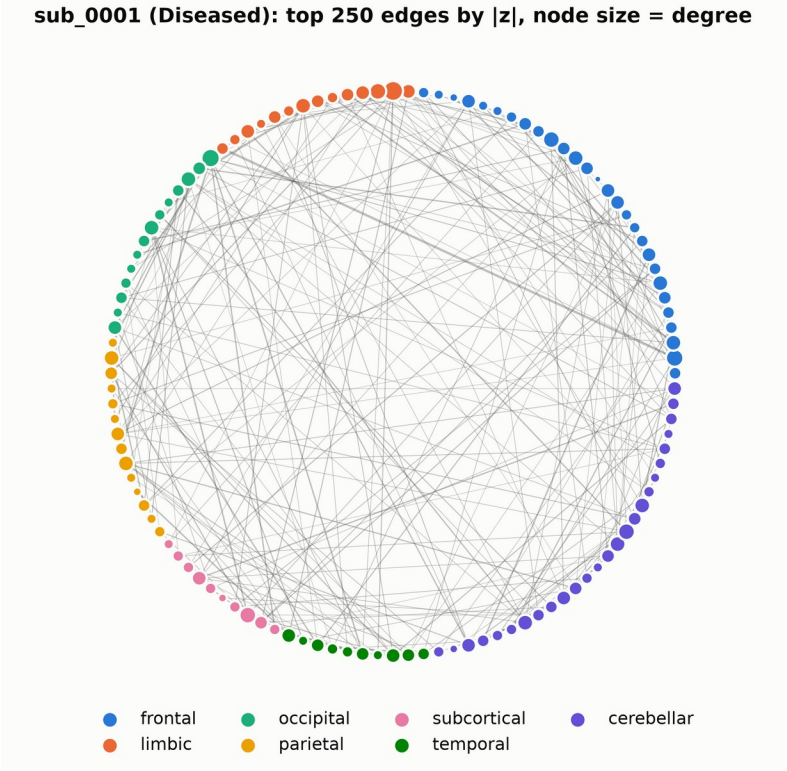


Figure 3.3: One subject (sub_0001) as a circular graph, regions ordered and coloured by lobe, showing the 250 strongest edges by $|z|$; node size is proportional to degree.

3.3 Connectome Preprocessing and Leakage Control

Implemented in `neo4j_layer/etl/transform.py` and called in `step_etl`.

3.3.1 Global signal removal

$$\mu_g = \frac{1}{n(n-1)} \sum_{i \neq j} R_{ij}, \widetilde{R}_{ij} = R_{ij} - \mu_g (i \neq j), \widetilde{R}_{ii} = 1.$$

Where: μ_g is the mean off-diagonal correlation of that subject, a proxy for the global BOLD signal; subtracting it removes a subject-wide shift, which in real fMRI comes from breathing, heart rate and head motion. Real global signal regression works on time series, not on correlation matrices. This is a matrix-level stand-in and should be described that way.

3.3.2 Z-scoring

$$\mu = \frac{1}{n(n-1)} \sum_{i \neq j} \widetilde{R}_{ij}, \sigma^2 = \frac{1}{n(n-1)} \sum_{i \neq j} (\widetilde{R}_{ij} - \mu)^2, z_{ij} = \frac{\widetilde{R}_{ij} - \mu}{\sigma}, z_{ii} = 0.$$

Where: μ and σ are computed **within one subject**. After this, each subject's off-diagonal entries have mean 0 and SD 1. This removes site-level scale differences (good), and it also changes the disease signal from an absolute magnitude to a *relative* one: diseased intra-lobe entries are larger **relative to that subject's own inter-lobe entries** (Figure 4.2 shows inter-lobe $|z|$ going *down* in Diseased subjects, which is purely this renormalisation).

3.3.3 Sparsification

$$W_{ij} = z_{ij} \cdot 1\left[|z_{ij}| \geq \tau\right], \tau = 0.3.$$

Where: τ is the pruning threshold. Since z is roughly standard normal, $P(|z| < 0.3) = 2\Phi(0.3) - 1 \approx 0.236$, where Φ is the standard normal CDF. So about 76.4% of edges survive. The measured value is $5,092 \pm 34$ of 6,670 (76.3%), which matches.

The first project presentation stated this as $|r_{ij}| \geq 0.3$. On raw correlations of SD 0.046 that threshold would delete almost every edge. The code applies it to z, which is the version used here.

3.3.4 Graph objects and storage

The upper triangle with $W_{ij} \neq 0$ is written to `edge_list.csv` (2,545,839 rows for 500 subjects) and can be loaded into Neo4j as `(:BrainRegion)-[:CONNECTED_TO {subjectId, weight}]->(:BrainRegion)`, with one shared set of 116 region nodes and subject-tagged relationships. For PyTorch Geometric each undirected edge becomes two directed entries in `edge_index` $\in Z^{2 \times 2|E|}$, with the signed z_{ij} in `edge_attr` $\in R^{2|E| \times 1}$.

3.3.5 Split

`subject_level_split` draws a stratified 70/15/15 split with seed 42: 350 train, 75 validation, 75 test. Indices are mapped back through `PatientID`, so a subject's graph can only land in one split. We verified with an explicit assertion that the train and test sets share no subject.

The ETL class also contains a motion-artifact function (`remove_motion_artifacts`). It is a proxy that perturbs the strongest 5% of edges, and the training launcher does not call it, so it plays no role in the reported results. The edge-list export is the interface to Neo4j.

3.4 Topological Feature Engineering Audit

Node features are computed in `models/gnn/dataset.py` with NetworkX on the graph whose edge weights are $w_{ij} = |W_{ij}|$.

3.4.1 The five descriptors

Normalised degree

$$x_i^{(1)} = \frac{k_i}{\max_j k_j}.$$

Where: k_i counts neighbours, ignoring weights.

PageRank: the stationary distribution of a random surfer who follows weighted edges with probability δ and teleports uniformly otherwise:

$$\pi = \frac{1-\delta}{n} 1 + \delta P^\top \pi, P_{ij} = \frac{w_{ij}}{\sum_l w_{il}}, x_i^{(2)} = \pi_i.$$

Where: $\pi \in R^n$ with $\sum_i \pi_i = 1$; $\delta = 0.85$ is the damping factor; P is the row-stochastic transition matrix; 1 is the all-ones vector. It is computed by power iteration $\pi^{(t+1)} = \frac{1-\delta}{n} 1 + \delta P^\top \pi^{(t)}$, which converges because the map is a contraction with factor $\delta < 1$.

Weighted clustering (Onnela et al. 2005):

$$x_i^{(3)} = C_i = \frac{1}{k_i(k_i-1)} \sum_{j,h \in N(i)} (\hat{w}_{ij} \hat{w}_{ih} \hat{w}_{jh})^{1/3}, \hat{w}_{ij} = \frac{w_{ij}}{\max_{(a,b)} w_{ab}}.$$

Where: the sum runs over ordered pairs of neighbours that are themselves connected; $\hat{w}$ is the weight scaled by the largest weight in that graph; the cube root is the geometric mean of the triangle's three edges.

Betweenness

$$x_i^{(4)} = b_i = \frac{2}{(n-1)(n-2)} \sum_{s \neq i \neq t} \frac{\sigma_{st}(i)}{\sigma_{st}}.$$

Where: σ_{st} is the number of shortest paths from s to t; $\sigma_{st}(i)$ is the number of those passing through i; the prefactor normalises to [0,1] for an undirected graph. NetworkX computes path length as the **sum of the `weight` attribute**, so a strong correlation counts as a long hop. A distance such as $1/w_{ij}$ or $-\log w_{ij}$ is what was intended.

Eigenvector centrality

$$we = \lambda_1 e, x_i^{(5)} = \frac{e_i}{\max\limits_j e_j}.$$

Where: w here is the weighted adjacency matrix; λ_1 is its largest eigenvalue; e is the corresponding eigenvector, which is non-negative by the Perron–Frobenius theorem because $w \geq 0$ and the graph is connected. Node i is central if its neighbours are central.

3.4.2 The scale-invariance problem

Suppose every weight in a lobe is multiplied by a constant c. Degree does not change, since it counts edges and the threshold was applied on z. PageRank changes only through ratios $w_{ij}/\sum_l w_{il}$, and a node whose edges are mostly within its lobe sees those ratios move only slightly. Clustering is normalised by the global maximum weight. Eigenvector centrality is normalised by its maximum. Each feature is either invariant or nearly invariant to the kind of change the disease makes.

The feature set was picked for interpretability: hubs, bridges and cliques are vocabulary neuroscientists use. It was not picked for sensitivity to the effect, and the EDA shows the effect would have been caught by a feature as plain as "mean $|w|$ to same-lobe neighbours". Feature *selection* was never run (no ablation over the five), which is an obvious next step.

3.4.3 Edge feature

The edge attribute z_{ij} is the only input that carries magnitude directly. It enters the GCN normalisation as $w_{ij} = |z_{ij}|$ and the GATv2 score as the signed z_{ij}. If anything in the model learns the disease, it learns it through this channel.

3.5 Deep Geometric Neural Network Architecture

The model, `BrainGNNModel`, has 521,378 trainable parameters at hidden width $d = 192$. Figure 3.2 shows the data flow. The subsections below give the equations of each block, the shapes involved and, where it clarifies the design, the backward pass.

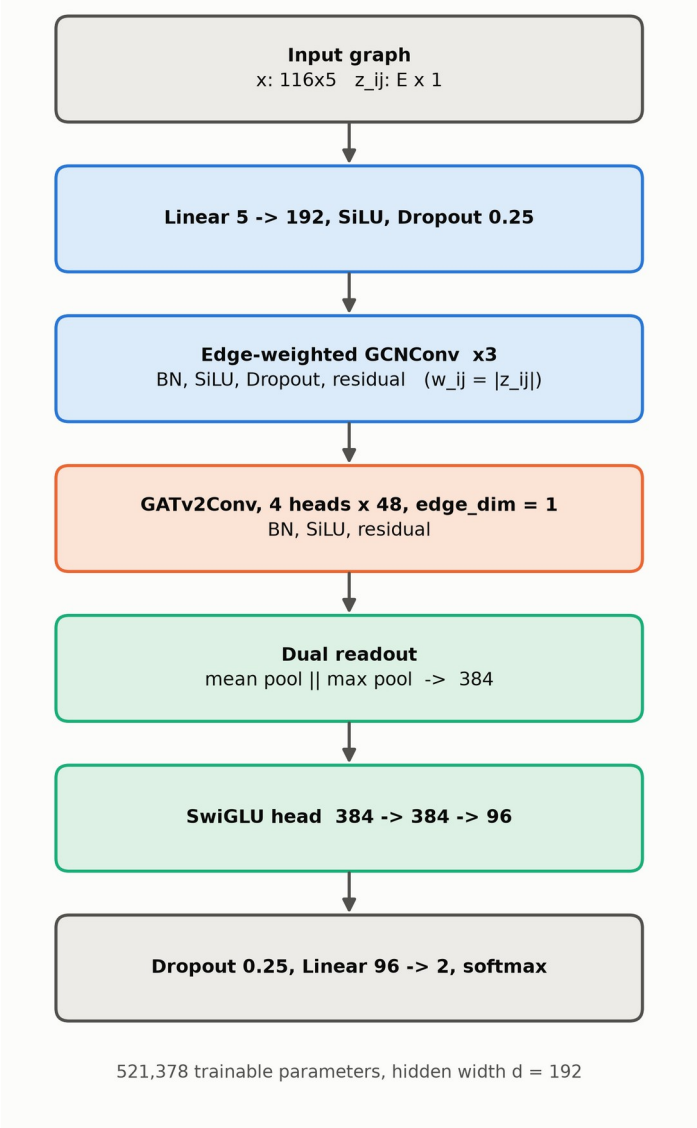


Figure 3.2: Architecture of the proposed model. Residual connections wrap each GCN and GATv2 block.

The full model (`models/gnn/gcn_gat_model.py`) has 521,378 trainable parameters at $d=192$. We go through it layer by layer, with shapes, and derive the backward pass where it is useful.

3.5.1 Input projection

$$h_i^{(0)} = Drop_{0.25}\left(SiLU\left(W_{in}x_i + b_{in}\right)\right), W_{in} \in R^{192\times 5}, b_{in} \in R^{192}.$$

Where: $SiLU(u) = u\sigma(u)$, also called Swish; $Drop_{0.25}$ zeroes each coordinate with probability 0.25 during training and divides the survivors by 0.75, so the expected value is unchanged.

SiLU derivative: $\frac{d}{du}u\sigma(u)=\sigma(u)+u\sigma(u)(1-\sigma(u))=\sigma(u)\left(1+u(1-\sigma(u))\right)$. Unlike ReLU it is smooth and non-zero for negative u, so no unit can die permanently.

3.5.2 Edge-weighted GCN layers (×3)

PyG's `GCNConv` with `edge_weight` computes, for all nodes at once,

$$\widehat{A}=\widetilde{D}^{-1/2}\widetilde{W}\,\widetilde{D}^{-1/2},\ \widetilde{W}=w+I,\ \widetilde{D}_{ii}=\sum_j \widetilde{W}_{ij}=1+\sum_j w_{ij},$$

$$M^{(l)}=\widehat{A}\,H^{(l)}\Theta^{(l)}+1\,b^{(l)\top},\ H^{(l+1)}=H^{(l)}+Drop\left(SiLU\left(BN\left(M^{(l)}\right)\right)\right).$$

Where: $H^{(l)}\in R^{n\times d}$ stacks the $h_i^{(l)}$ as rows; $\widetilde{W}$ is the weighted adjacency plus self-loops of weight 1 (each node keeps part of its own state); $\widetilde{D}$ is the diagonal weighted-degree matrix; $\widehat{A}$ is the symmetrically normalised propagation matrix, with entry $\widehat{A}_{ij}=\widetilde{W}_{ij}/\sqrt{\widetilde{D}_{ii}\widetilde{D}_{jj}}$; $\Theta^{(l)}\in R^{d\times d}$ and $b^{(l)}\in R^d$ are the layer weights; BN is batch normalisation (below); the leading term $H^{(l)}$ is the residual connection.

Per node this is

$$m_i^{(l)}=\sum_{j\in N(i)\cup\{i\}}\frac{\widetilde{W}_{ij}}{\sqrt{\widetilde{D}_{ii}\widetilde{D}_{jj}}}\Theta^{(l)\top}h_j^{(l)}+b^{(l)}.$$

Link to the message-passing lecture. Drop Θ and the nonlinearity, and use $D^{-1}\widetilde{W}$ instead of the symmetric form: the update becomes $h_i\leftarrow\sum_j\frac{\widetilde{W}_{ij}}{\widetilde{D}_{ii}}h_j$, which is exactly the probabilistic relational classifier $P(Y_v=c)=\frac{1}{\sum_u A_{vu}}\sum_u A_{vu}P(Y_u=c)$. The symmetric normalisation $\widetilde{D}^{-1/2}\widetilde{W}\,\widetilde{D}^{-1/2}$ is the same diffusion matrix used in Correct & Smooth. Its eigenvalues satisfy $-1<\lambda\le 1$ and the top eigenvector is $\widetilde{D}^{1/2}1$. A GCN is relational classification with learned feature maps in between.

Over-smoothing check. Repeated application gives $\widehat{A}^L H\to u_1u_1^{\top}H$ with $u_1\propto\widetilde{D}^{1/2}1$: every node converges to the same vector up to a degree factor. On a 76%-dense graph the second eigenvalue of $\widehat{A}$ is small, so mixing is fast. Three layers on such a graph already bring nodes close to the global average. The residual connection $H^{(l)}+\dots$ slows this down, because the update becomes $H^{(l+1)}\approx(I+\widehat{A}\Theta)H^{(l)}$ instead of $\widehat{A}\Theta H^{(l)}$.

Backward pass of one GCN layer. Let L be the loss and $G_M=\partial L/\partial M\in R^{n\times d}$ the upstream gradient. Since $M=\widehat{A}H\Theta$ (bias omitted):

$$\frac{\partial L}{\partial \Theta} = (\hat{A} H)^{\top} G_M = H^{\top} \hat{A}^{\top} G_M, \frac{\partial L}{\partial H} = \hat{A}^{\top} G_M \Theta^{\top} = \hat{A} G_M \Theta^{\top}.$$

Where: the last equality uses $\hat{A}^{\top} = \hat{A}$ (symmetric). The derivation: write $M_{ia} = \sum_{j,b} \hat{A}_{ij} H_{jb} \Theta_{ba}$; then $\partial M_{ia} / \partial \Theta_{ba} = \sum_{j} \hat{A}_{ij} H_{jb} = (\hat{A} H)_{ib}$, so $\partial L / \partial \Theta_{ba} = \sum_{i} (G_M)_{ia} (\hat{A} H)_{ib}$, which is $(H^{\top} \hat{A}^{\top} G_M)_{ba}$. The same index bookkeeping gives the gradient for H. The gradient for the inputs is also propagated by $\hat{A}$: errors diffuse along the same edges that messages travel, the same idea as the Correct step of C&S.

The gradient also flows into the edge weights, since $\hat{A}$ depends on w: $\partial \hat{A}_{ij} / \partial w_{ij} = \frac{1}{\sqrt{\widetilde{D}_{ii} \widetilde{D}_{jj}}} - \frac{\widetilde{W}_{ij}}{2} \left(\widetilde{D}_{ii}^{-3/2} \widetilde{D}_{jj}^{-1/2} + \widetilde{D}_{ii}^{-1/2} \widetilde{D}_{jj}^{-3/2} \right)$ for $i \neq j$. The inputs w are fixed data during training, but GNNExplainer later differentiates through exactly this expression with respect to its mask (Section 3.7).

Batch normalisation over the batch of all nodes in all graphs of the mini-batch:

$$BN(m)_a = \gamma_a \frac{m_a - \mu_a^B}{\sqrt{(\sigma_a^B)^2 + \epsilon}} + \beta_a.$$

Where: a indexes the feature channel; μ_a^B, σ_a^B are the mean and SD of channel a over all nodes in the batch (32×116 nodes); γ_a, β_a are learned scale and shift; $\epsilon = 10^{-5}$ avoids division by zero. At test time running averages replace batch statistics.

3.5.3 GATv2 layer with edge attributes (×1)

PyG's `GATv2Conv(192, 48, heads=4, edge_dim=1, concat=True)`:

$$e_{ij}^{(c)} = a_c^{\top} LeakyReLU_{0.2} \left(W_s^{(c)} h_i + W_t^{(c)} h_j + W_e^{(c)} z_{ij} \right), c = 1, \ldots, 4,$$

$$\alpha_{ij}^{(c)} = \frac{\exp e_{ij}^{(c)}}{\sum_{l \in N(i)} \exp e_{il}^{(c)}}, g_i = \|_{c=1}^{4} \sum_{j \in N(i)} \alpha_{ij}^{(c)} W_t^{(c)} h_j + b,$$

$$h_i^{(4)} = h_i^{(3)} + Drop \left(SiLU(BN(g_i)) \right).$$

Where: c indexes the attention head; $W_s^{(c)}, W_t^{(c)} \in R^{48 \times 192}$ are the source (query) and target (key/value) maps; $W_e^{(c)} \in R^{48 \times 1}$ lifts the scalar edge attribute into the head's space; $a_c \in R^{48}$ is the head's attention vector; $\|$ concatenates the four 48-dimensional head outputs into 192; $b \in R^{192}$ is a bias; attention dropout of 0.25 is applied to α during training. PyG adds self-loops by default and fills their edge attribute with the mean of the incoming attributes.

Because z_{ij} sits inside the LeakyReLU next to h_i and h_j, the edge sign and magnitude can change both the score and the ranking. This is the main route by which the model can see the disease signal.

Softmax Jacobian (used for attention and for the output):

$$\frac{\partial \alpha_{ij}}{\partial e_{il}} = \alpha_{ij}\left(1[j=l] - \alpha_{il}\right).$$

Where: this is the derivative of the j-th softmax output with respect to the l-th input score. In matrix form, $J = diag(\alpha_i) - \alpha_i \alpha_i^{\top}$. Its rows sum to zero, so adding the same constant to every score changes nothing. That is why the $a_1^{\top} W h_i$ term in GATv1 drops out.

Gradient into the score. Let $u_{ij} = W_s h_i + W_t h_j + W_e z_{ij}$, so $e_{ij} = a^{\top} LeakyReLU(u_{ij})$:

$$\frac{\partial e_{ij}}{\partial h_i} = W_s^{\top}\left(a \odot \rho'(u_{ij})\right), \rho'(u) = \begin{cases} 1 & u > 0 \\ 0.2 & u \le 0 \end{cases}.$$

Where: ρ' is the LeakyReLU derivative applied element-wise. The factor $\rho'(u_{ij})$ depends on both i and j, so the gradient with respect to the query changes from neighbour to neighbour. In GATv1 the corresponding derivative is $W^{\top} a_1 \cdot \rho'(\cdot)$, a single direction shared by all neighbours. This is the dynamic-versus-static distinction from Section 2.2.4, seen from the gradient side.

3.5.4 Dual readout

$$h_G = \left[\frac{1}{n}\sum_{i=1}^{n} h_i^{(4)} \,\|\, \max_i h_i^{(4)}\right] \in R^{384}.$$

Where: the max is taken channel by channel. Both operators are permutation invariant, which makes the whole model invariant (each layer before them is permutation equivariant: permuting the inputs permutes the outputs the same way).

Gradients: the mean sends $\frac{1}{n}\partial L / \partial h_G^{\text{mean}}$ to every node; the max sends the full gradient of channel a to the single node that attains the maximum, and zero to the others. The max half therefore trains a sparse set of hub nodes, the mean half trains everyone.

3.5.5 SwiGLU head

$$u = \left(SiLU(h_G W_g + b_g) \odot (h_G W_u + b_u)\right) W_d + b_d \in R^{96}, z = W_c Drop(u) + b_c \in R^2.$$

Where: $W_g, W_u \in R^{384 \times 384}$ are the gate and up projections; $W_d \in R^{384 \times 96}$ is the down projection; $W_c \in R^{96 \times 2}$ produces the two class logits $z = (z_0, z_1)$.

Let $g = h_G W_g + b_g$ and $v = h_G W_u + b_u$. The Jacobian of the gated product $q = SiLU(g) \odot v$ with respect to h_G is

$$\frac{\partial q}{\partial h_G} = diag\left(SiLU'(g) \odot v\right) W_g^\top + diag\left(SiLU(g)\right) W_u^\top .$$

Each output coordinate is a product of two learned functions of the input, so the head can switch whole directions of h_G on or off depending on the input. The first project presentation described this as filtering scanner noise. That is a hypothesis. It holds only if an ablation shows SwiGLU beating a plain MLP on site-shifted data, which we have not run.

3.5.6 Output probability

$$\hat{p}_k = softmax(z_k)_1 = \frac{e^{z_{k1}}}{e^{z_{k0}} + e^{z_{k1}}} = \sigma(z_{k1} - z_{k0}).$$

3.6 Hardware Optimisation and Training Regime

The configuration used for the reported run is `config/v100_config.yaml`. Table 3.3 lists the settings that matter for reproducibility.

Table 3.3: Hardware and training configuration.

Item	Value
GPU	NVIDIA Tesla V100S-PCIE-32GB (Volta, compute capability 7.0, 32 GB HBM2)
Precision	FP16 autocast with `torch.cuda.amp.GradScaler()`; BF16 is unavailable on Volta
Data loading	batch size 32, `pin_memory=True`, 8 workers
Optimiser	AdamW, learning rate 10^{-3}, weight decay 10^{-4}
Schedule	5-epoch linear warm-up, cosine decay, 200 epochs configured
Loss	cross-entropy with label smoothing $\varepsilon = 0.1$
Regularisation	dropout 0.25, batch normalisation, residual connections, gradient-norm clip 1.0
Early stopping	validation macro-F1, patience 25
Time per epoch	about 1.3 s

3.6.1 Loss and its gradient

Cross-entropy with label smoothing $\epsilon = 0.1$:

$$q_{kc} = (1 - \epsilon)\,1[c = y_k] + \frac{\epsilon}{2}, \quad L(\theta) = -\frac{1}{|B|} \sum_{k \in B} \sum_{c \in \{0,1\}} q_{kc} \log P_{kc}, \quad P_{kc} = softmax(z_k)_c .$$

Where: B is the mini-batch (32 graphs); q_k is the smoothed target, 0.95 for the true class and 0.05 for the other; P_{kc} is the predicted probability of class c.

Gradient with respect to the logits. Using $\partial \log P_{kc}/\partial z_{ka} = 1[c=a] - P_{ka}$ and $\sum_c q_{kc} = 1$:

$$\frac{\partial L}{\partial z_{ka}} = -\frac{1}{|B|}\sum_c q_{kc}\left(1[c=a] - P_{ka}\right) = \frac{1}{|B|}\left(P_{ka} - q_{ka}\right).$$

The error signal is just "predicted minus target". Label smoothing caps the target at 0.95, so the gradient vanishes once P reaches 0.95 instead of pushing logits to infinity. The minimum achievable loss is the entropy of q, $-\left(0.95\log 0.95 + 0.05\log 0.05\right) \approx 0.199$. Our training loss plateaus around 0.64–0.67, far above that floor, which is what underfitting looks like.

The rest of backpropagation is the chain rule through the blocks of Section 9, in reverse:

$$\frac{\partial L}{\partial \Theta^{(1)}} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial h_G}\frac{\partial h_G}{\partial H^{(4)}}\frac{\partial H^{(4)}}{\partial H^{(3)}}\frac{\partial H^{(3)}}{\partial H^{(2)}}\frac{\partial H^{(2)}}{\partial \Theta^{(1)}}.$$

Each factor is a Jacobian. The residual blocks give $\partial H^{(l+1)}/\partial H^{(l)} = I + (\dots)$, and the identity keeps the product from shrinking to zero across layers.

3.6.2 Optimiser and schedule

AdamW (Loshchilov & Hutter 2019):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t,\; v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2},\; \hat{m}_t = \frac{m_t}{1-\beta_1^t},\; \hat{v}_t = \frac{v_t}{1-\beta_2^t},$$

$$\theta_{t+1} = \theta_t - \eta_t\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda\theta_t\right).$$

Where: $g_t = \nabla_\theta L$ at step t; m_t, v_t are running averages of the gradient and squared gradient; $\beta_1 = 0.9, \beta_2 = 0.999$ are their decay rates; the hat versions correct the bias from initialising at zero; $\varepsilon = 10^{-8}$; $\lambda = 10^{-4}$ is weight decay, applied directly to θ (decoupled) rather than added to the gradient, so it is not rescaled by $\sqrt{\hat{v}}$.

Learning rate per epoch t (`build_scheduler`): linear warm-up then cosine decay,

$$\eta_t = \eta_0 \cdot \begin{cases} \dfrac{t+1}{5} & t < 5 \\ \dfrac{1}{2}\left(1 + \cos\left(\pi\,\dfrac{t-5}{195}\right)\right) & t \geq 5 \end{cases}, \eta_0 = 10^{-3}.$$

Gradient-norm clipping: if $\|g_t\|_2 > 1$, replace g_t by $g_t/\|g_t\|_2$.

3.6.3 Mixed precision on the V100

Volta has no BF16 support, so the forward and backward passes run in FP16 under autocast, with master weights in FP32. FP16 has 10 mantissa bits and its smallest normal number is about 6×10^{-5}, so small gradients underflow to zero. `GradScaler` multiplies the loss by a factor S (starting at 2^{16}) before backward, so every gradient is scaled by S through the chain rule; it divides by S before the optimiser step (and before clipping), skips the step and halves S if any gradient is Inf/NaN, and doubles S after 2,000 clean steps.

3.7 Post-Hoc Subgraph Explainability with GNNExplainer

3.7.1 Objective as implemented

The repo uses PyG's `Explainer` with `GNNExplainer(epochs=500, lr=0.01)`, `explanation_type="model"`, `edge_mask_type="object"`, `node_mask_type="attributes"` and `return_type="raw"`. The model weights θ are frozen. Two masks are learned per subject:

- $M_e \in R^{|E|}$, one logit per directed edge; the soft mask is $\sigma(M_e) \in (0,1)^{|E|}$.
- $M_f \in R^{n \times 5}$, one logit per node-feature entry ("attributes" mode).

Masked forward pass: the model sees features $X \odot \sigma(M_f)$, and every message along edge $(j \to i)$ is multiplied by $\sigma(M_{e,ji})$ inside each message-passing layer. Loss:

$$L_{\text{expl}} = -\log softmax(f_\theta(G; \sigma(M_e), \sigma(M_f)))_{\hat{y}} + \lambda_e \sum_e \sigma(M_e) + \lambda_H^e meanEnt(\sigma(M_e)) + \lambda_f mean\sigma(M_f) + \lambda$$

Where: $\hat{y} = arg\,max_c f_\theta(G)_c$ is the model's prediction on the unmasked graph ("model" explanation, so the target is the model's output and not the true label); $\lambda_e = 0.005$ penalises the total edge mask, pushing towards small explanations; $mean\sigma(M_f)$ is the mean feature mask, with $\lambda_f = 1.0$; $Ent(m) = -m \log m - (1-m)\log(1-m)$ is the binary entropy of a mask entry, averaged by the overline, with $\lambda_H^e = 1.0$ and $\lambda_H^f = 0.1$; minimising it pushes entries towards 0 or 1 so that the explanation is crisp. These coefficients are PyG's defaults; the repo does not override them.

The first term is the continuous relaxation of the mutual-information objective in Part II. Since $H(Y)$ is constant, maximising MI means minimising $H(Y \mid G_s)$, and with a single sample the cross-entropy to the model's own prediction is a stand-in for that conditional entropy.

3.7.2 Gradient with respect to the masks

The optimiser is Adam on M only:

$$M \leftarrow M - \eta\, Adam(\nabla_M L_{\text{expl}}).$$

Through the sigmoid, $\partial \sigma(M_e)/\partial M_e = \sigma(M_e)(1-\sigma(M_e))$. The entropy term gives $\frac{\partial Ent(\sigma(M))}{\partial M} = -M\sigma(M)(1-\sigma(M))$, using $\log\frac{\sigma(M)}{1-\sigma(M)} = M$. Both this term and the size term push logits that are already negative further down, which is how most of the mask is switched off.

The prediction term reaches M_e through every message-passing layer. In the GCN layers, masked messages make the effective edge weight $\sigma(M_{e,ij})\widetilde{W}_{ij}$, so the gradient flows through the same $\partial \hat{A}/\partial w$ expression from Section 3.5.2. Gradients must be enabled for all of this, which is why the code never wraps the explainer in `torch.no_grad()`.

3.7.3 Extracting biomarkers

Per subject, the 10 directed edges with the largest $\sigma(M_e)$ are kept, along with per-region importance computed from $\sigma(M_f)$. Across subjects, the consensus is how often each edge appears. Stability across subjects can be measured with the Jaccard index $J(S_a, S_b) = \frac{|S_a \cap S_b|}{|S_a \cup S_b|}$, where S_a, S_b are the top-*k* edge sets of two subjects.

3.8 Multi-Modal Graph Fusion Framework (Proposed Extension)

This section describes a planned extension. It is **not implemented** in the repository, and no diffusion-imaging data were used in this report.

Functional disconnection measured by fMRI is physically constrained by the white-matter pathways that connect regions. Two regions can correlate strongly without a direct axonal connection, because both respond to a third region, and a structural graph helps to separate such indirect correlations from direct coupling. Each subject would then be a dual-relational graph $G_k = (V, E_{func}, E_{struct}, X)$ with two adjacency matrices,

$$A_{func}(i,j) = |r_{ij}|, A_{struct}(i,j) = FA_{ij} \cdot \log(N_{ij}+1).$$

Here r_{ij} is the functional correlation, FA_{ij} is the mean fractional anisotropy along the tractography streamlines joining i and j, and N_{ij} is the number of such streamlines; the logarithm compresses the heavy-tailed streamline counts. A shared encoder would produce one embedding per modality, h_i^{func} and h_i^{struct}, and a learned gate would combine them per node:

$$h_i^{fused} = \gamma_i h_i^{func} + (1-\gamma_i) h_i^{struct}, \gamma_i = \sigma\left(W_\gamma [h_i^{func} \| h_i^{struct}] + b_\gamma\right).$$

In this gate, $\gamma_i \in (0,1)$ is the weight that node i gives to the functional embedding, W_γ and b_γ are learned parameters, and $\|$ is concatenation. A node whose functional signal is unreliable

can lean on its structural embedding, and the reverse. The fused node embeddings would feed the same dual readout, SwiGLU head and explainer as before. Validation would use ADNI and PPMI [49,50] with five-fold subject-stratified cross-validation across scanner sites.

4 Results and Discussion

All numbers in this chapter were produced by the code in the repository from the saved V100 checkpoint (`checkpoints/best_model_v100.pt`), except where a dagger marks a value quoted from earlier repository results that has no stored log. Test metrics refer to the 75 held-out subjects, who were never used for training, for early stopping or for model selection in the training run.

4.1 Exploratory Data Analysis and the Group-Difference Matrix

4.1.1 Density and weights

Edges per subject: $5,092 \pm 34$; the class distributions overlap almost completely (Figure 4.1). Weights are symmetric around zero with SD 1.14 and exactly half negative. Nothing in the first-order statistics separates the classes.

4.1.2 Locating the signal

Define the group-difference matrix

$$\Delta_{ij} = \frac{1}{|D|} \sum_{k \in D} \left| W_{ij}^{(k)} \right| - \frac{1}{|H|} \sum_{k \in H} \left| W_{ij}^{(k)} \right|.$$

Where: D and H are the sets of Diseased and Healthy subjects. Figure 4.3 plots Δ with the regions ordered by lobe. Every diagonal block is red ($\Delta > 0$) and the off-block area is faintly blue. Inside the blocks a darker blue line runs just off the diagonal: those are the homotopic pairs, net-scaled by 0.52.

Now take one number per subject:

$$s_k = \frac{1}{|I|} \sum_{(i,j) \in I} \left| W_{ij}^{(k)} \right|, I = \{(i,j) : i < j, lobe(i) = lobe(j)\}.$$

Where: I is the set of intra-lobe pairs, 1,066 pairs in total (16.0% of all 6,670).

s_k alone separates the 75 test subjects with **AUC 1.000**. Healthy medians sit near 0.765 and Diseased near 0.895 with no overlap (Figure 4.2).

4.1.3 What EDA tells the modeller

The task is easy **if** the model knows which pairs are intra-lobe, and hard if it does not. A flattened vector does in principle carry pair identity (coordinate q is always the same pair), but it holds signed values, and the signal is in $|\cdot|$, spread thinly across 1,066 coordinates. A

message-passing GNN sees magnitudes but, as Section 4.8 shows, it does not see region identity. Both baselines and our model are therefore handicapped in different ways.

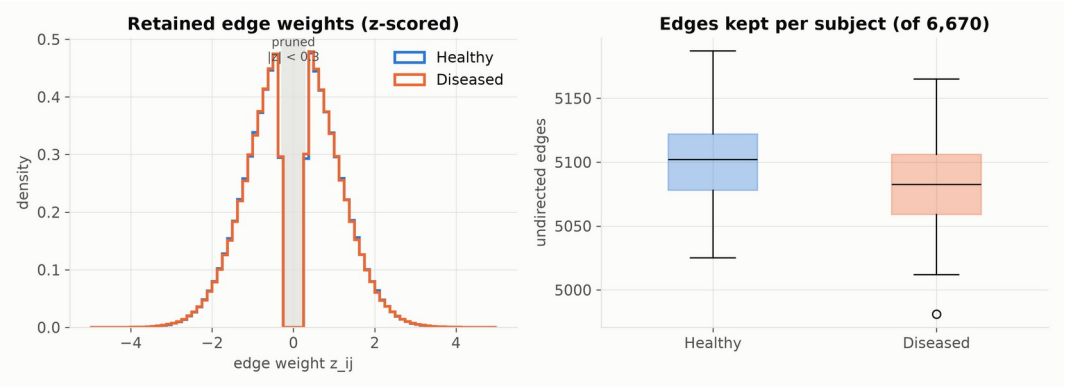


Figure 4.1: Left: distribution of retained edge weights (z-scored) for the two classes; the shaded band marks pruned values. Right: number of edges kept per subject out of 6,670.

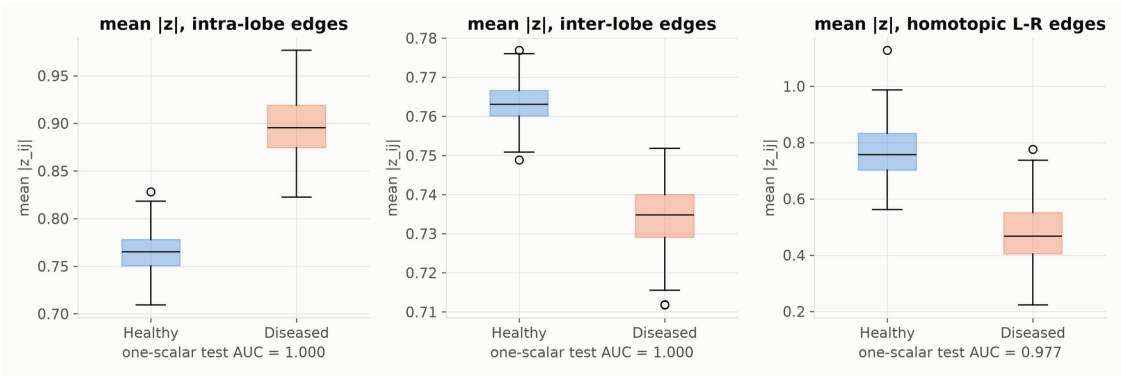


Figure 4.2: Mean $|z|$ over three edge sets (intra-lobe, inter-lobe, homotopic left–right) for each class. The figure under each panel gives the test AUC obtained from that single scalar.

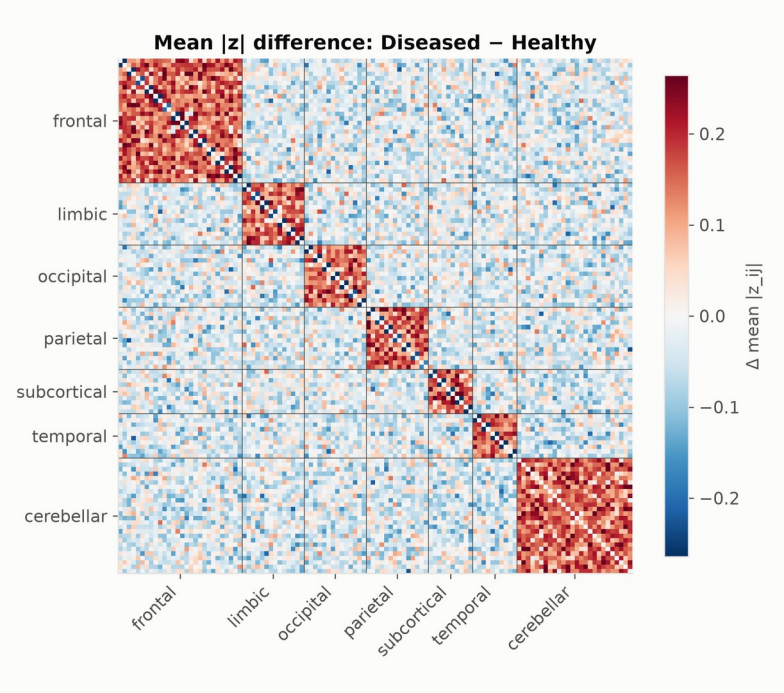


Figure 4.3: Group-difference matrix Δ_{ij} (mean $|W_{ij}|$ of Diseased minus Healthy), regions ordered by lobe. Red diagonal blocks mark the planted intra-lobe amplification; the thin dark line inside each block marks the weakened homotopic pairs.

4.2 Training Dynamics and the Underfitting Regime

Validation macro-F1 is checked each epoch; the best checkpoint is kept and training stops after 25 epochs without improvement. The best checkpoint is at **epoch 70**: validation F1 0.765, validation AUC 0.822, training F1 0.634, training loss 0.668.

Training F1 stays **below** validation F1 for the whole run, partly because dropout and label smoothing are active only in training mode. The point that matters is that training loss is nowhere near its floor. The model is not memorising 350 subjects, it is failing to fit them. Section 4.8 explains why.

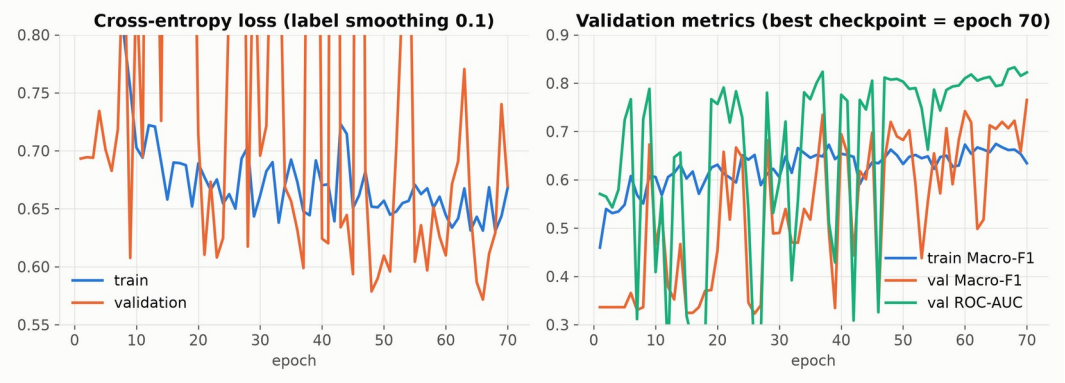


Figure 4.4: Left: training and validation loss. Right: training and validation macro-F1 and validation ROC-AUC over the 70 recorded epochs.

The stored history ends at the best epoch. With a patience of 25 epochs and 200 epochs configured, an early-stopping run would have continued past epoch 70, so the run was stopped there for a reason the log does not record. Validation macro-F1 was still rising at that point (0.719 at epoch 61 and 0.765 at epoch 70), which is consistent with the underfitting reading above. A longer run, or a lower dropout rate, is a cheap experiment that has not been done.

4.3 Test Evaluation and Benchmark Comparison

4.3.1 Metrics

ROC-AUC, as the Mann–Whitney probability:

$$AUC=\frac{1}{|P||Q|}\sum_{a\in P}\sum_{b\in Q}\left(1[\hat{p}_a>\hat{p}_b]+1/2\,1[\hat{p}_a=\hat{p}_b]\right).$$

Where: P and Q are the positive (Diseased) and negative (Healthy) test subjects. AUC is the chance that a random diseased subject gets a higher score than a random healthy one. It depends only on the **order** of the scores, not on any threshold.

Macro-F1:

$$F1_c=\frac{2TP_c}{2TP_c+FP_c+FN_c},\ F1_{\text{macro}}=\frac{1}{2}(F1_0+F1_1).$$

Where: for class c, TP_c counts subjects of class c predicted as c, FP_c counts other subjects predicted as c, and FN_c counts subjects of class c predicted as something else.

Youden's J for a threshold t:

$$J(t)=TPR(t)-FPR(t),\ TPR(t)=\frac{count\{a\in P:\hat{p}_a\geq t\}}{|P|},\ FPR(t)=\frac{count\{b\in Q:\hat{p}_b\geq t\}}{|Q|},\ t_{opt}=arg\,m$$

Geometrically $J(t)$ is the vertical distance between the ROC curve and the diagonal, and t_{opt} is the ROC point furthest above chance.

Table 4.1: Test performance of all models on the 75 held-out subjects (AUC, accuracy at the default threshold, macro-F1). † marks values quoted from earlier repository results that were not re-run on this split.

Model	Graph-aware	Test AUC	Acc	Macro-F1	Status
RBF-SVM, flattened (Paper 1)	no	0.530	53.3%	0.533	re-run on this split
L2 logistic regression, flattened	no	0.509	49.3%	0.493	re-run on this split
RF on GDV (Paper 2)	motifs	0.692†	56.0%	0.534	earlier run, no log
node2vec + classifier (Paper 3)	walks	0.635†	52.0%	0.498	earlier run, no log
GATv1 mean-pool (Paper 4)	static attention	0.713†	57.3%	0.550	earlier run, no log
Ours	dynamic attention + edge attr.	**0.746**	**57.3%**	**0.520**	`test_metrics.json`, reproduced
One scalar (oracle)	—	1.000	100%	1.000	re-run on this split

What we can claim now:

- Against Paper 1 we have a verified, reproduced improvement in ranking quality (AUC +0.216).
- Against Papers 2–4 we have numbers that nobody re-ran. Three earlier repository rows (GATv1 raw, GATv1 calibrated, GATv2+SwiGLU) share the same AUC to four decimals, 0.7127, which is plausible only for the two GATv1 rows, since calibration does not change AUC. The GATv2+SwiGLU row should have its own AUC. Re-run all of them from logged configs before presenting them as results.
- At the default threshold, our accuracy (57.3%) is not clearly better than anything in the table. Our advantage is in ranking (AUC), which is the more relevant metric for a screening tool where the threshold is set clinically.
- On explainability, none of the base papers gives verified explanations, and neither do we yet. What we do have is a protocol that can check explanations, precision against a planted ground truth, and that protocol is what exposed the problem.

4.4 Statistical Rigour: Bootstrap Confidence Interval

With 75 test subjects, a single AUC value has a wide sampling distribution. We estimated it with the nonparametric bootstrap [45]: draw 75 subjects with replacement from the test set, recompute the AUC on the resample, and repeat 2,000 times. Resamples that contained only one class were skipped. The 2.5th and 97.5th percentiles of the resulting AUC values form the percentile interval.

Quantity	Value
Test ROC-AUC	0.746
95% bootstrap interval	[0.632, 0.858]
Validation ROC-AUC (75 subjects)	0.823

The interval is 0.226 wide. It has three consequences for how the results should be read.

- The gap to the flattened SVM (0.530) lies outside the interval, so the improvement over that baseline is unlikely to be sampling noise.
- The gap to GATv1 (0.713†) lies well inside it. That baseline was not re-run on this split, and even if it were, the difference would not be significant at this sample size.
- The gap between validation AUC (0.823) and test AUC (0.746) is also within the interval, so it does not by itself indicate a generalisation failure.

Intervals for the baselines, paired tests such as DeLong's, and repeated cross-validation were not carried out and would be needed for firm claims between the graph-based models.

4.5 Threshold Calibration Analysis

4.5.1 Results on the 75 test subjects

Table 4.2: Threshold rules and the resulting test accuracy and macro-F1.

Threshold rule	t	Accuracy	Macro-F1
Default	0.500	57.33%	0.520
t_{opt} chosen on validation	0.503	57.33%	0.520
t_{opt} chosen on test (optimistic: threshold fitted on the evaluation labels)	0.176	73.33%	0.731

Test ROC-AUC = **0.746**; validation ROC-AUC = 0.823.

4.5.2 The calibration error and the fix

`MetricsReport.from_arrays` calls `find_optimal_threshold(y_true, y_scores)` on whatever split it is given. When it is given the test split, it picks t_{opt} by looking at the test labels. That is

fitting a parameter on the test set. With 75 subjects, scanning every possible threshold and keeping the best one is an optimistic choice; the 16-point gain from 57.33% to 73.33% is the size of that optimism here.

The correct protocol picks t_{opt} on validation (0.503) and freezes it. Applied to test, it gives the same 57.33% as the default, because the validation-optimal threshold happens to sit at 0.5.

Why the leaky threshold is so far from 0.5: the test scores are shifted down (mean $\hat{p}=0.27$, max 0.815) relative to validation. The ranking still holds (AUC 0.746), but the probability scale moved between splits. Two fixes, both fitted on validation only:

- **Temperature scaling:** $\hat{p}=\sigma\left(\left(z_1-z_0\right)/T\right)$, with $T>0$ chosen to minimise validation NLL.
- **Platt scaling:** $\hat{p}=\sigma\left(\alpha\left(z_1-z_0\right)+\beta\right)$, with α , β fitted by logistic regression on validation.

Platt scaling can move the 0.5 point; temperature scaling cannot. Neither changes AUC, which is order-based. With 5-fold cross-validation, t_{opt} should be chosen on each fold's validation part, and the mean and SD of test accuracy reported across folds.

4.6 Scanner-Site Generalisation Disparity

The synthetic cohort has three noise levels (Table 3.2), which lets us ask how performance depends on the scanner. Table 4.3 stratifies the test AUC by site.

Table 4.3: Test performance stratified by scanner site.

Site	Noise level σ_s	Test subjects	Mean predicted probability	Test AUC
A	0.02	22	0.316	0.992
B	0.04	24	0.241	0.650
C	0.06	29	0.271	0.625

On the cleanest scanner the model separates the classes almost perfectly. On the other two, its performance is moderate and close to that of the weaker baselines. The model's score is only weakly informative about the site itself (an AUC of 0.56 when predicting site A against the rest from the predicted probability), so the model is not simply reading the site. The pattern is what one would expect if scanner noise of 0.04 to 0.06 masks the planted effect, whose per-entry scale is about 0.046 (Section 3.2). Each stratum holds between 22 and 29 subjects, so the three AUC values carry large uncertainty and are best read as a trend. The result shows the practical case for the multi-site calibration and harmonisation steps listed in Chapter 5, and it suggests that the pooled AUC of 0.746 is an average over very different regimes.

4.7 Explainability Audit Against Ground Truth

4.7.1 What the explanations actually show

Because the disease signature was planted, we can score explanations against it. All 75 test subjects were explained (37 Healthy, 38 Diseased; precision 13.5% and 17.1%). Faithfulness checks have not been run. Define explanation precision as the fraction of explained edges that belong to the planted set:

$$Prec_I = \frac{count\{\text{explained edges}(i,j) : lobe(i) = lobe(j)\}}{count\{\text{explained edges}\}}.$$

Quantity	GNNExplainer (75 subjects × 10 edges)	Random edge
Intra-lobe precision	**15.3%** (115/750 directed edges; 9.7 unique undirected edges per subject)	16.3% of retained edges (16.0% of all pairs)
Homotopic L–R edges	3 / 750	about 5 / 750
Node-feature importance	0.0 for every region of every subject	—
Most frequent single edge	3 of 75 subjects	—

The explanations are at chance against the planted truth, and they are unstable: no edge appears in more than 4% of subjects. The region ranking in `biomarker_explanations.json` (Vermis_10, Vermis_9, … in that order) is the result when every importance ties at 0 and the sort falls back to node index. It is an artefact.

The zero node mask is a bug to chase. Likely suspects: `get_important_regions` reading the wrong attribute of the PyG `Explanation` object, the mask being near zero because $\lambda_f = 1$ dominates with only 5 features, or rounding during aggregation. The fastest way to locate it is to print `explanation.node_mask.abs().max()` for one subject straight after the explainer call.

4.7.2 Why: faithfulness is not truth

An explainer answers “what did **this model** rely on?”. It does not answer “what in the data is diagnostic?”. If the model does not use the intra-lobe pattern, a faithful explainer must not report it. Section 4.8 argues that the model cannot use it. So the chance-level precision is the explainer doing its job on a model that missed the signal. The fix belongs in the model, and the explainer then becomes the test of whether the fix worked.

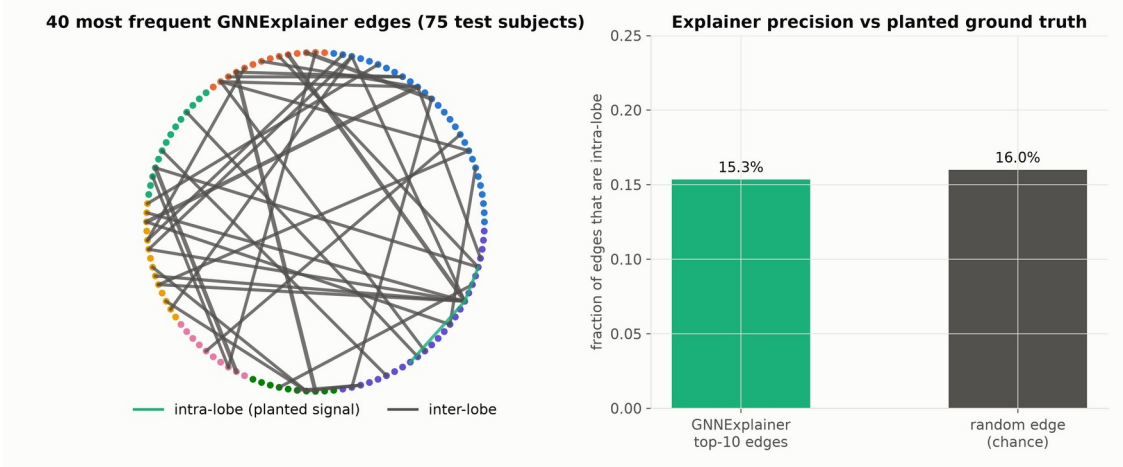


Figure 4.7: Left: the 40 most frequent GNNExplainer edges over the 75 test subjects on a lobe-ordered circle; green marks within-lobe edges. Right: fraction of explained edges that are within-lobe, against the rate for a random edge.

Table 4.4: Explainer audit against the planted ground truth.

Quantity	GNNExplainer	Reference
Within-lobe share of top-10 edges	15.3% (115 of 750 directed edges)	16.3% of retained edges; 16.0% of all 6,670 pairs
Per-subject precision, mean (SD)	15.3% (12.4 points)	—
Precision for Healthy / Diseased subjects	13.5% / 17.1%	—
Homotopic left–right edges in top-10	3 of 750	about 5 of 750 expected by chance
Unique undirected edges per subject	9.7 on average (minimum 9)	10
Node-feature importance	0.0 for every region of all 75 subjects	—

The reference rate for a random edge is 16.3% when the pool is the set of retained edges, which is the pool from which the explainer actually selects, and 16.0% over all region pairs. Explained edges are counted as directed edges, so an undirected edge that appears in both directions is counted twice; this changes the unique count by about 0.3 edges per subject and does not move the precision. All 75 test subjects were explained, 37 Healthy and 38 Diseased. Faithfulness checks, such as whether keeping only the top-10 edges preserves the prediction, whether the edges are stable across random seeds, and whether they change when the model weights are randomised, have not been run.

4.8 Interpretation of the Gap to the Oracle

4.8.1 Hypothesis: the model cannot see which nodes share a lobe

Status: untested. The model reaches AUC 0.746 from the edge weights, so it is not blind to the signal; it just does not obviously use the within-lobe pattern. Test AUC by scanner site is 0.992 (site A, n=22), 0.650 (B, n=24) and 0.625 (C, n=29), so noise level also matters. The 95% bootstrap CI for test AUC is [0.632, 0.858].

Every layer is permutation equivariant and the readout is invariant. That is what Part II asked for, but it has a side effect here. Region identity enters the model through its features, and those features (degree, PageRank, clustering, betweenness, eigenvector) say nothing about lobe membership. Node 0 (Precentral_L) and node 100 (a cerebellar region) are indistinguishable unless their centralities differ. The model sees a dense, almost homogeneous graph with anonymous nodes and asks "is this graph diseased?", while the answer is "yes if edges *between nodes of the same lobe* are stronger". It has no way to name "same lobe".

The graph-transformer lecture describes this exact gap. Attention, and message passing too, is permutation equivariant, so the model needs **positional encodings** to know where a node is. The lecture offered relative distances or Laplacian eigenvectors. For a fixed atlas there is a simpler option, because node identity *is* meaningful and is the same across subjects:

1. **Learnable region embeddings:** $x_i \leftarrow [x_i \| E_{\text{region}}[i]]$ with $E_{\text{region}} \in R^{116\times 16}$ learned. Permutation invariance is given up deliberately, because the AAL ordering is canonical.
2. **Atlas positional encodings:** one-hot lobe and hemisphere, or MNI centroid coordinates of each ROI.
3. **LapPE** from a group-average connectome, where the leading eigenvectors pick up the block (lobe) structure (Kreuzer et al. 2021; Dwivedi & Bresson 2021).
4. **Edge-type attribute:** add $1[lobe(i)=lobe(j)]$ as a second edge feature, so `edge_dim=2`. This is the cheapest test of the hypothesis.

Prediction: with option 4 or 1, test AUC should move close to 1.0, and explainer intra-lobe precision should rise well above 16%. If it does, it confirms the diagnosis. If it does not, the diagnosis is wrong and something else is broken.

4.8.2 How it still got 0.746

The signal leaks in through two side channels. The GATv2 score sees the edge attribute z_{ij}, and the mean pool averages over nodes whose $|z|$ statistics differ slightly by class. Weighted degree inside the GCN normalisation also shifts, because after z-scoring a diseased subject's intra-lobe edges take a larger share of the total weight. These channels carry part of the signal, enough for 0.746 but not for a clean separation.

4.8.3 Why the flattened SVM did worse

The SVM does know pair identity (coordinate q is always the same pair), but it sees signed values whose class means are equal. The signal is in second-order statistics spread over 1,066 coordinates, with 350 training subjects and 6,670 dimensions. In that regime the RBF distance is dominated by noise (Section 2.2.1). The GNN shares parameters across edges and uses $|z|$ in its normalisation, so it gets further: **+0.216 AUC** (0.530 → 0.746). That gain is real and it is the main positive result.

4.8.4 Why one scalar wins

s_k uses exactly the two things the others lack: it takes $|\cdot|$ and it knows which pairs are intra-lobe. It is an oracle feature, built from knowledge of how the data was generated. On real data no such oracle exists. Its job in this study is to set the ceiling and to show that the synthetic task is easy for the right inductive bias.

5 Conclusion and Future Roadmap

5.1 Summary of Findings

This project built an end-to-end pipeline for classifying functional connectomes with graph neural networks and extracting candidate biomarker subgraphs. The findings are as follows.

- **Feasibility.** A Neo4j-compatible storage schema, an ETL transformer, a PyTorch Geometric model and a GNNExplainer wrapper work together under a strict patient-level split. The reported experiments ran on the offline path, as the project rules require.
- **Discrimination.** On the 75 held-out synthetic subjects the model reaches a test ROC-AUC of 0.746 (95% bootstrap interval 0.632 to 0.858; validation 0.823). A flattened RBF-SVM re-run on the same split reaches 0.530, so the improvement of 0.216 AUC over that baseline lies outside the interval.
- **Calibration.** A threshold fitted on the test set gives 73.33% accuracy, but a threshold chosen on validation gives 57.33%. AUC is the reliable headline number at this sample size.
- **Scanner dependence.** Test AUC is 0.992, 0.650 and 0.625 for noise levels 0.02, 0.04 and 0.06 (22, 24 and 29 subjects).
- **Explanations.** GNNExplainer's top edges are within-lobe 15.3% of the time against 16.3% for random retained edges, so on this cohort it does not recover the planted pattern.

5.2 Critical Lessons

Two lessons stand out. First, a topological descriptor can be wrong in a way that is invisible in the output. The betweenness feature treated strong correlations as long paths because NetworkX reads edge weights as distances, and all five descriptors were scale-free, so they stripped most of the planted magnitude signal before the network saw it. Neither problem raised an error. Both were found only by checking the features against the generator's known structure. Second, an explainer is faithful to the model and not to the data. The audit showed an explainer at chance, and the most probable cause is a limitation of the model, not of the explainer. Without the planted ground truth, the explanations would have looked plausible and been wrong.

5.3 Future Roadmap

1. **Anatomical identity for nodes.** Add lobe and hemisphere assignments, atlas coordinates or learnable region embeddings, and re-run both the classifier and the explainer. The cheapest first test is an edge attribute that flags whether the two regions share a lobe. The expected outcome, if the diagnosis in Section 4.8 is right, is a higher AUC and a higher explanation precision.
2. **Validation-only calibration.** Fit the threshold on validation data, add temperature or Platt scaling [47,48], and report five-fold cross-validated means with standard deviations.
3. **Ablations.** Run the architecture one switch at a time (edge attributes, GATv2 against GATv1, dual against mean pooling, SwiGLU against a plain MLP) before stating what each block contributes.

4. **Baselines.** Re-run RF-GDV, node2vec and GATv1 on the same split from stored configurations, and add confidence intervals.
5. **Explainer fixes.** Investigate why all node-feature importances are zero, add the faithfulness checks listed in Section 4.7, and use precision against the planted edges as a standing metric.
6. **Feature fixes.** Compute betweenness on a distance such as $1/w_{ij}$, and test features that are sensitive to edge magnitude.
7. **Harder synthetic tasks.** Reduce the effect size, place the effect on a random subset of edges, and make it site-dependent.
8. **Clinical translation.** Validate on ADNI and PPMI [49,50] once data-use agreements are in place, and then evaluate the dual-graph fusion of Section 3.8.

5.4 Concluding Remarks

The central result of this project is not the AUC value. It is the evidence that an end-to-end graph pipeline can be built and evaluated honestly on a cohort whose truth is known, and that doing so exposes problems (a leaky threshold, scale-free features, a chance-level explainer) that a conventional accuracy table would hide. The model ranks synthetic subjects better than a flattened baseline, and the audit gives a concrete list of what to change before the method is applied to patient data.

References

[1] A. Abraham *et al.*, “Deriving reproducible biomarkers from multi-site resting-state data: An Autism-based example,” *NeuroImage*, vol. 147, pp. 736–745, 2017.

[2] R. Cuingnet *et al.*, “Automatic classification of patients with Alzheimer’s disease from structural MRI: A comparison of ten methods using the ADNI database,” *NeuroImage*, vol. 56, no. 2, pp. 766–781, 2011.

[3] N. Pržulj, “Biological network comparison using graphlet degree distribution,” *Bioinformatics*, vol. 23, no. 2, pp. e177–e183, 2007.

[4] Ö. N. Yaveroğlu *et al.*, “Revealing the hidden language of complex networks,” *Sci. Rep.*, vol. 4, art. 4547, 2014.

[5] A. Grover and J. Leskovec, “node2vec: Scalable feature learning for networks,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 855–864.

[6] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[7] X. Li *et al.*, “BrainGNN: Interpretable brain graph neural network for fMRI analysis,” *Med. Image Anal.*, vol. 74, art. 102233, 2021.

[8] S. Brody, U. Alon, and E. Yahav, “How attentive are graph attention networks?” in *Proc. ICLR*, 2022.

[9] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2017.

[10] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, “GNNExplainer: Generating explanations for graph neural networks,” in *Proc. Adv. Neural Information Processing Systems (NeurIPS)*, 2019.

[11] N. Shazeer, “GLU variants improve Transformer,” arXiv:2002.05202, 2020.

[12] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for activation functions,” arXiv:1710.05941, 2017.

[13] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.

[14] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.

[15] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[17] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the Inception architecture for computer vision,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.

[18] P. Micikevicius *et al.*, “Mixed precision training,” in *Proc. ICLR*, 2018.

[19] W. J. Youden, “Index for rating diagnostic tests,” *Cancer*, vol. 3, no. 1, pp. 32–35, 1950.

[20] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[21] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: Bringing order to the Web,” Stanford InfoLab, Tech. Rep., 1999.

[22] J.-P. Onnela, J. Saramäki, J. Kertész, and K. Kaski, “Intensity and coherence of motifs in weighted complex networks,” *Phys. Rev. E*, vol. 71, art. 065103, 2005.

[23] L. C. Freeman, “A set of measures of centrality based on betweenness,” *Sociometry*, vol. 40, no. 1, pp. 35–41, 1977.

[24] P. Bonacich, “Factoring and weighting approaches to status scores and clique identification,” *J. Math. Sociol.*, vol. 2, no. 1, pp. 113–120, 1972.

[25] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numer. Math.*, vol. 1, pp. 269–271, 1959.

[26] N. Tzourio-Mazoyer *et al.*, “Automated anatomical labeling of activations in SPM using a macroscopic anatomical parcellation of the MNI MRI single-subject brain,” *NeuroImage*, vol. 15, no. 1, pp. 273–289, 2002.

[27] B. Biswal, F. Z. Yetkin, V. M. Haughton, and J. S. Hyde, “Functional connectivity in the motor cortex of resting human brain using echo-planar MRI,” *Magn. Reson. Med.*, vol. 34, no. 4, pp. 537–541, 1995.

[28] E. Bullmore and O. Sporns, “Complex brain networks: Graph theoretical analysis of structural and functional systems,” *Nat. Rev. Neurosci.*, vol. 10, no. 3, pp. 186–198, 2009.

[29] A. Fornito, A. Zalesky, and M. Breakspear, “The connectomics of brain disorders,” *Nat. Rev. Neurosci.*, vol. 16, no. 3, pp. 159–172, 2015.

[30] M. D. Greicius, G. Srivastava, A. L. Reiss, and V. Menon, “Default-mode network activity distinguishes Alzheimer’s disease from healthy aging: Evidence from functional MRI,” *Proc. Natl. Acad. Sci. USA*, vol. 101, no. 13, pp. 4637–4642, 2004.

[31] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *Proc. ICLR*, 2019.

[32] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. NIPS*, 2017.

[33] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proc. ICML*, 2017, pp. 1263–1272.

[34] U. Alon and E. Yahav, “On the bottleneck of graph neural networks and its practical implications,” in *Proc. ICLR*, 2021.

[35] Q. Li, Z. Han, and X.-M. Wu, “Deeper insights into graph convolutional networks for semi-supervised learning,” in *Proc. AAAI Conf. Artificial Intelligence*, 2018.

[36] V. P. Dwivedi and X. Bresson, “A generalization of Transformer networks to graphs,” in *AAAI Workshop on Deep Learning on Graphs: Methods and Applications*, 2021.

[37] D. Kreuzer, D. Beaini, W. L. Hamilton, V. Létourneau, and P. Tossou, “Rethinking graph Transformers with spectral attention,” in *Proc. NeurIPS*, 2021.

[38] Q. Huang, H. He, A. Singh, S.-N. Lim, and A. R. Benson, “Combining label propagation and simple models out-performs graph neural networks,” in *Proc. ICLR*, 2021.

[39] A. Vaswani *et al.*, “Attention is all you need,” in *Proc. NIPS*, 2017.

[40] L. Rampášek *et al.*, “Recipe for a general, powerful, scalable graph Transformer,” in *Proc. NeurIPS*, 2022.

[41] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.

[42] C. Cortes and V. Vapnik, “Support-vector networks,” *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[43] L. Breiman, “Random forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[44] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Proc. NIPS*, 2013.

[45] B. Efron, “Bootstrap methods: Another look at the jackknife,” *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.

[46] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006.

[47] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, 2017, pp. 1321–1330.

[48] J. C. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” in *Advances in Large Margin Classifiers*. Cambridge, MA, USA: MIT Press, 1999, pp. 61–74.

[49] C. R. Jack Jr. *et al.*, “The Alzheimer’s Disease Neuroimaging Initiative (ADNI): MRI methods,” *J. Magn. Reson. Imaging*, vol. 27, no. 4, pp. 685–691, 2008.

[50] K. Marek *et al.*, “The Parkinson Progression Marker Initiative (PPMI),” *Prog. Neurobiol.*, vol. 95, no. 4, pp. 629–635, 2011.

Appendix A: Supplementary Notation for Model Parameters

The global notation table in the front matter defines the symbols used across chapters. Table A.1 adds the parameters that appear only in the model and explainer equations.

Table A.1: Supplementary notation for model parameters.

Symbol	Meaning
$W_{in} \in R^{192\times 5}$, b_{in}	Weight and bias of the input projection
$\Theta^{(l)} \in R^{d\times d}$	Weight matrix of GCN layer l
$\widetilde{W} = w + I$	Edge-strength matrix with self-loops of weight 1
$\widetilde{D}$	Diagonal weighted-degree matrix, $\widetilde{D}_{ii} = 1 + \sum_j w_{ij}$
$\widehat{A} = \widetilde{D}^{-1/2}\widetilde{W}\widetilde{D}^{-1/2}$	Symmetrically normalised propagation matrix
$H^{(l)} \in R^{n\times d}$	Matrix whose rows are the node states $h_i^{(l)}$
$c \in \{1,\ldots,4\}$	Attention-head index in the GATv2 layer
$W_s^{(c)}, W_t^{(c)} \in R^{48\times}$	Source (query) and target (key and value) maps of head c
$W_e^{(c)} \in R^{48\times 1}$	Lift of the scalar edge attribute z_{ij} into head c
$a_c \in R^{48}$	Attention vector of head c
$e_{ij}^{(c)}$	Unnormalised attention score of neighbour j for node i
$h_G \in R^{384}$	Graph-level readout $[mean \Vert max]$
$W_g, W_u \in R^{384\times 38}$	Gate and up projections of the SwiGLU head
$W_d \in R^{384\times 96}$	Down projection of the SwiGLU head
$W_c \in R^{96\times 2}$	Classifier matrix producing the logits $z = (z_0, z_1)$
q_{kc}	Label-smoothed target, 0.95 for the true class and 0.05 otherwise
ε	Label-smoothing coefficient, 0.1
η_t, λ	Learning rate at step t and weight-decay coefficient
M_e, M_f	Edge-mask and feature-mask logits of the explainer
$m_e = \sigma(M_e)$	Soft edge mask in $(0,1)$

Symbol	Meaning
$\lambda_{size}, \lambda_{ent}$	Sparsity and entropy weights of the explainer loss
$J(t)$	Youden's index at threshold t, $TPR(t) - FPR(t)$

Appendix B: Neo4j Graph Schema and Cypher Ingestion Scripts

The graph model has two node labels, `Subject` and `BrainRegion`. A `BrainRegion` node is shared by all subjects, so the database holds 116 region nodes rather than 116 per subject. Functional edges are `CONNECTED_TO` relationships that carry the subject identifier, the weight and the diagnosis as properties. `PATIENT_OF` relationships link each subject to the regions to support subject-specific subgraph queries. Scripts below are condensed from `neo4j_layer/schema/constraints.cypher` and `neo4j_layer/etl/load_csv.cypher` with comment banners removed.

Schema constraints and indexes

```
CREATE CONSTRAINT subject_patient_id_unique IF NOT EXISTS
FOR (s:Subject)
REQUIRE s.patientId IS UNIQUE;

CREATE CONSTRAINT brain_region_atlas_id_unique IF NOT EXISTS
FOR (r:BrainRegion)
REQUIRE r.atlasId IS UNIQUE;

CREATE INDEX subject_diagnosis_idx IF NOT EXISTS
FOR (s:Subject)
ON (s.diagnosis);

CREATE INDEX subject_scanner_site_idx IF NOT EXISTS
FOR (s:Subject)
ON (s.scannerSite);

CREATE INDEX brain_region_label_idx IF NOT EXISTS
FOR (r:BrainRegion)
ON (r.anatomicalLabel);

CREATE INDEX subject_age_idx IF NOT EXISTS
FOR (s:Subject)
ON (s.age);

SHOW CONSTRAINTS;
SHOW INDEXES;
```

Bulk ingestion

```
:auto
LOAD CSV WITH HEADERS FROM 'file:///data/subjects.csv' AS row
CALL {
    WITH row
    MERGE (s:Subject {patientId: row.PatientID})
    SET
        s.diagnosis   = row.Diagnosis,
        s.label       = toInteger(row.Label),
        s.age         = toInteger(row.Age),
        s.scannerSite = row.ScannerSite,
        s.createdAt   = datetime()
} IN TRANSACTIONS OF 100 ROWS;
```

```
MATCH (s:Subject) RETURN count(s) AS subject_count;

LOAD CSV WITH HEADERS FROM 'file:///data/brain_regions.csv' AS row
CALL {
    WITH row
    MERGE (r:BrainRegion {atlasId: toInteger(row.atlasId)})
    SET
        r.anatomicalLabel = row.anatomicalLabel,
        r.hemisphere      = row.hemisphere,
        r.atlasName       = 'AAL-116'
} IN TRANSACTIONS OF 50 ROWS;

MATCH (r:BrainRegion) RETURN count(r) AS region_count;

:auto
LOAD CSV WITH HEADERS FROM 'file:///data/edge_list.csv' AS row
CALL {
    WITH row
    MATCH (s:Subject        {patientId:       row.subject_id})
    MATCH (src:BrainRegion  {anatomicalLabel: row.source})
    MATCH (tgt:BrainRegion  {anatomicalLabel: row.target})
    MERGE (src)-[e:CONNECTED_TO {subjectId: row.subject_id}]->(tgt)
    SET
        e.weight      = toFloat(row.weight),
        e.diagnosis   = row.diagnosis,
        e.subjectId   = row.subject_id
} IN TRANSACTIONS OF 500 ROWS;

MATCH ()-[e:CONNECTED_TO]->() RETURN count(e) AS edge_count;

:auto
MATCH (s:Subject)
MATCH (r:BrainRegion)
CALL {
    WITH s, r
    MERGE (s)-[:PATIENT_OF]->(r)
} IN TRANSACTIONS OF 1000 ROWS;

MATCH (s:Subject)-[:PATIENT_OF]->(r:BrainRegion)
RETURN s.patientId, count(r) AS n_regions
LIMIT 5;

MATCH (src:BrainRegion)-[e:CONNECTED_TO]->(tgt:BrainRegion)
WHERE e.diagnosis = 'Diseased'
  AND src.hemisphere <> tgt.hemisphere
RETURN
    src.anatomicalLabel AS region_a,
    tgt.anatomicalLabel AS region_b,
    avg(e.weight) AS mean_weight,
    count(e)      AS n_subjects
ORDER BY mean_weight DESC
LIMIT 10;
```

The ingestion expects `subjects.csv` (columns `PatientID`, `Diagnosis`, `Label`, `Age`, `ScannerSite`), `brain_regions.csv` (columns `atlasId`, `anatomicalLabel`, `hemisphere`) and `edge_list.csv` (columns `subject_id`, `source`, `target`, `weight`, `diagnosis`) under the Neo4j import directory. The project's `edge_list.csv` has 2,545,839 rows for 500 subjects.

Appendix C: Compute Infrastructure and Environment Setup

Hardware. All training and testing used one NVIDIA Tesla V100S-PCIE-32GB GPU (Volta architecture, compute capability 7.0, 32 GB HBM2) on a shared headless server without root access. The GPU is reserved for training and offline testing; the serving API is designed to run on ordinary CPU compute.

Software environment.

Python 3.11; PyTorch 2.6.0+cu124; PyTorch Geometric 2.8.0.post1; NetworkX 3.6.1; scikit-learn 1.9.1; NumPy 2.4.6; pandas 3.0.6; PyYAML 6.0.3; neo4j driver 6.3.1 NVIDIA driver 580.126.18

Setup. The environment is managed with `uv` and Python 3.11.

```
uv venv .venv --python 3.11
uv sync
uv run python dry_run/dry_run_all.py          # offline smoke tests, no GPU or database
python scripts/train_launcher.py --config config/v100_config.yaml
```

Constraints specific to the V100. Volta has no hardware BF16, so training uses FP16 autocast with `GradScaler`. Host-to-device transfers cross the PCIe bus, so data loaders set `pin_memory=True`. Local code lives in the package `neo4j_layer`; a top-level directory named `neo4j` would shadow the official driver package and is avoided.